

In[139]:= ClearAll["Global`*"]

Risk limiter functions

Summary

Soft knee downward limiter function

Soft knee downward limiter function, version 1

$$h(x; b, c) = \begin{cases} f(x; b, c) = \frac{1}{\exp((\log(b)(c x - 1)))} + (\vartheta c b \log(b) + 1) x & , \text{ for } x < -\frac{1}{c \log(b)} \equiv a \\ g(x) = x & , \text{ for } x \geq -\frac{1}{c \log(b)} \equiv a \end{cases}$$

$$f'(x) = d - c \log(b) b^{1-c x}$$

$$d = \vartheta c b \log(b) + 1$$

Constraints

The constraints on the downward limiting multiplier are:

Constraints on x

$x \geq 0$ (because x is a standard deviation)

Constraints on b

$0 < b < 1$

Constraints on c

$$0 < c < \frac{1}{b \log(b) (\vartheta - b^{-c x_{\text{extreme}}})}, x_{\text{extreme}} > -\frac{1}{c \log(b)} \equiv a$$

Equivalent:

$$0 < c < \frac{1}{b \log(b) (\vartheta - k_{\text{extreme}})}, k_{\text{extreme}} > \vartheta$$

$$k_{\text{extreme}} = b^{-c x_{\text{extreme}}} \Leftrightarrow x_{\text{extreme}} = -\frac{\log(k_{\text{extreme}})}{c \log(b)}$$

x_{extreme} is the maximum possible value of x . Theoretically this value is ∞ . In practice, this value is chosen heuristically. The trade-off is that the higher the x_{extreme} , the more narrow the valid range of c . For $x_{\text{extreme}} = \infty$ there will be no valid value of c . For values of x_{extreme} very close to ϑ , the max value of c can get very big, and the resulting $h(x)$ may have negative slope for some values of x .

For b values close to 1, c can get very big.

For implementation it can be a good idea to impose a maximum c value. Something like $c = 10$ might be reasonable.

For implementation, we don't know the minimum value of x_{extreme} , when we calculated the constraints on c . Instead we can calculate the constraints on c based on k_{extreme} , and then check that $x_{\text{extreme}} > -\frac{1}{c \log(b)}$.

Plot

Note: In the following plots, for hard inequalities we set the range of c to $[\delta, c_{\max}]$, and the range of b to $[\delta, b_{\max} - \delta]$.

```
In[653]:=  $\delta = 1 \cdot 10^{-8}$ ;
bmax = 1;
cmax = 10;
binit = 0.1;
cinit = 0.1;
kinit = N[E + 2];
(*xtreme=E+2;*) (*The higher the max x, the smaller the range of c*)

Clear[x, a, b, c]

(*aval[b_,c_,d_]:=1/c-log (d/(c log (b)))/(c log (b))**)
(*aval[x_,b_,c_]:=
  Solve[Log[b]*(1-c*x)-Log[x]==Log[-E*b*c Log[b]],x][[1]][[2]]*)
aval[b_,  $\gamma$ _] := -1 / ( $\gamma$  Log[b]);
(*tauzero[b_,  $\gamma$ _] :=  $\gamma$  * b*Log[b]*E;*)
(*tauinf[xtreme_, b_,  $\gamma$ _] :=  $\gamma$  * b*Log[b]*(E - b^(- $\gamma$ *xtreme))**)
tauinf[kextreme_,  $\beta$ _,  $\gamma$ _] :=  $\gamma$  *  $\beta$  * Log[b] * (E - kextreme)

f[x_,  $\beta$ _,  $\gamma$ _] := 1 / (Exp[Log[b] * ( $\gamma$  * x - 1)]) + (E *  $\gamma$  *  $\beta$  * Log[b] + 1) * x;
g[x_] := x;
h[x_,  $\beta$ _,  $\gamma$ _] :=
  Piecewise[{{f[x,  $\beta$ ,  $\gamma$ ], x < aval[b,  $\gamma$ ]}, {g[x], x >= aval[b,  $\gamma$ ]}}];

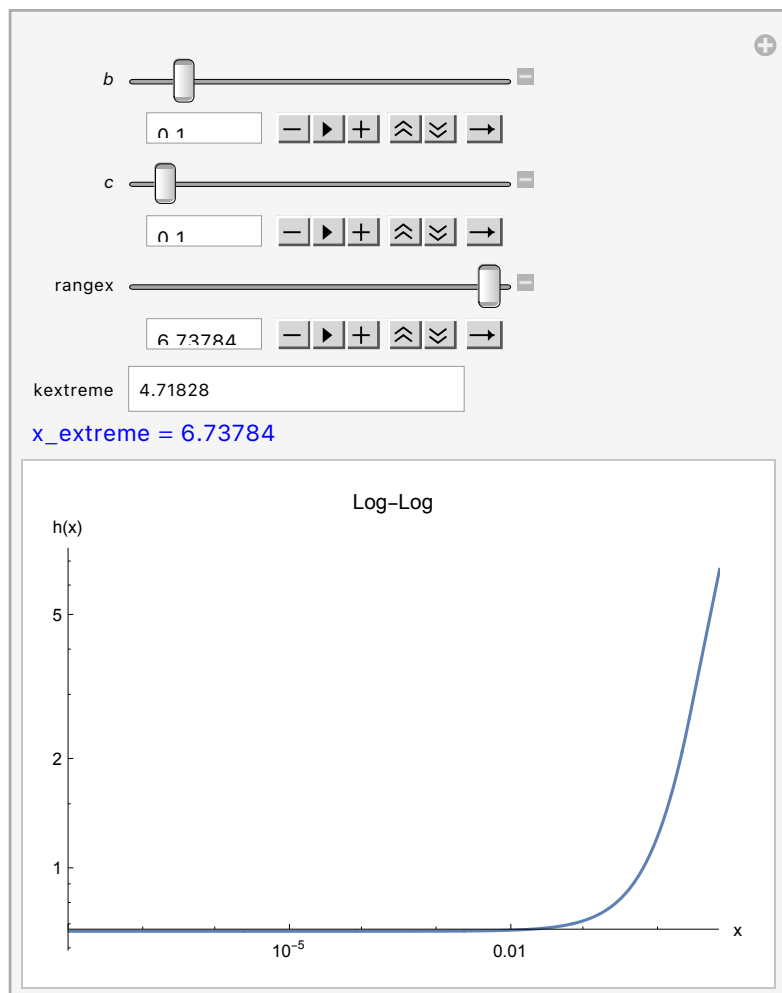
Manipulate[
  LogLogPlot[h[x, b, c],
    {x,  $\delta$ , rangex},
    AxesLabel -> {"x", "h(x)"},
    PlotLabel -> "Log-Log",
    PlotRange -> {{ $\delta$ , rangex}, Full}
  ],
  {{b, binit},  $\delta$ , bmax -  $\delta$ , Appearance -> "Open"},
  (*{{c, 0.1},  $\delta$ , If[b>1, 1/(b *Log[b]* E), cmax],Appearance->"Open"}*)
```

```

(*{{c, 0.1}, δ, If[b>1, NSolve[tauzero[b, γ]==1, γ, Reals][[1]][[2]],
  NSolve[tauinfn[b, γ]==1, γ, Reals][[1]][[2]], Appearance->"Open"]*)
(*{{c, 0.1}, δ, Min[1/(b*Log[b]*(E-xtreme)), cmax], Appearance->"Open"], *)
{c, cinit}, δ,
  Min[NSolve[tauinfn[kextreme, b, γ] == 1, γ, PositiveReals][[1]][[2]], cmax],
  Appearance->"Open"],
(*{{rangex, xtreme}, 2*δ, xtreme, Appearance->"Open"], *)
{{rangex, -Log[kinit] / (cinit*Log[binit])},
  2*δ, -Log[kextreme] / (c*Log[b]), Appearance->"Open"},
(*{xtreme, N[E+2]}*)
{kextreme, kinit},
Dynamic[Style["x_extreme = " <> TextString[N[-Log[kextreme] / (c*Log[b])]],
  12, If[kextreme > E, Blue, Red]]]
(*{{c, 1/(2*Log[b]*(b-1))}, δ, Min[maxc, 1/(Log[b]*(b-1))-δ],
  Appearance->"Open"]*)
]

```

Out[665]=



Downward limiter function, version 2a

$$f(x; b) = \frac{1}{\exp(\log(b)(cx-1))}$$

$$c = -\frac{1}{e b \log(b)}$$

$$g(x; b) = \begin{cases} f(x) & \text{for } x < a \equiv e b \\ x & \text{for } x > a \equiv e b \end{cases}$$

$$h(x; b, \lambda) = \lambda g(x; b) + (1 - \lambda) \max(x, b), \quad 0 \leq \lambda \leq 1$$

Constraints

Constraints on x

$x \geq 0$ (because x is a standard deviation)

Constraints on b

$0 < b < 1$

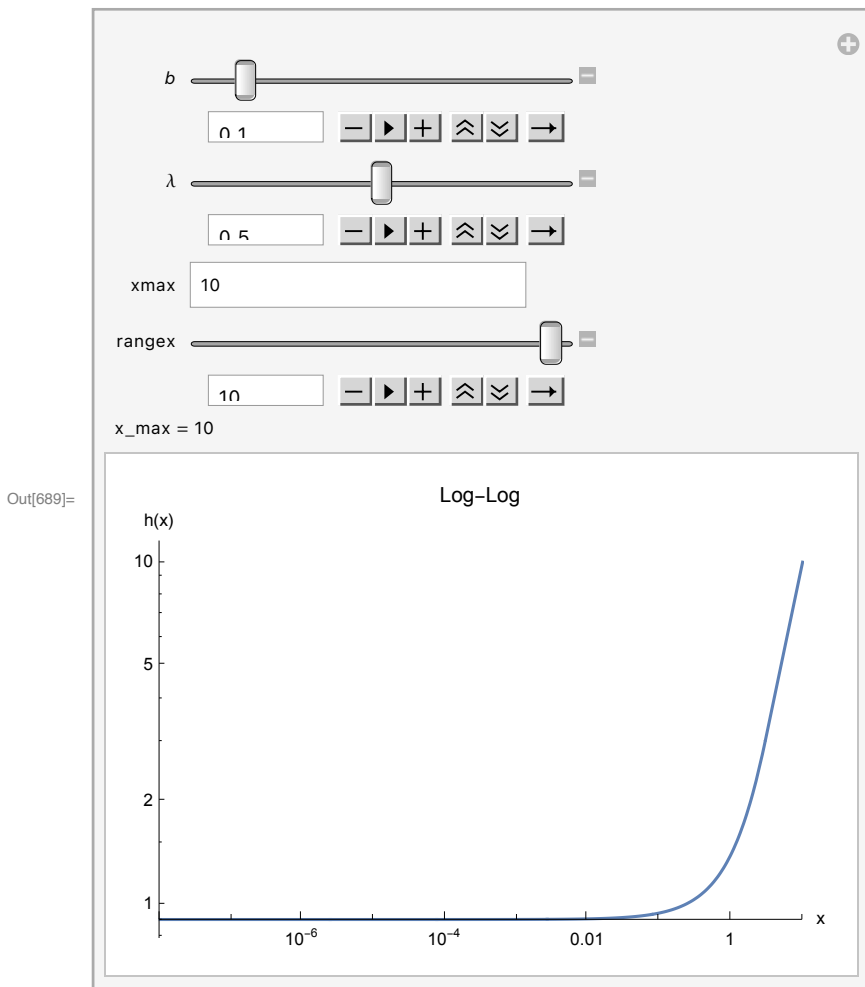
Plot

```
In[682]:=  $\delta = 1 \cdot 10^{-8};$ 
xmaxinit = 10;
lambdainit = 0.5;

Clear[x, a, b]

f[x_, b_] := 1 / (Exp[Log[b] * ((- 1 / (E * b * Log[b])) * x - 1)]);
g[x_, b_] := Piecewise[
  {{f[x, b], x < E b},
  {x, x ≥ E b}}
];
h[x_, b_, λ_] := λ * g[x, b] + (1 - λ) * Max[b, x]

Manipulate[
  LogLogPlot[h[x, b, λ],
    {x, δ, rangex},
    AxesLabel → {"x", "h(x)"},
    PlotLabel → "Log-Log",
    PlotRange → {{δ, rangex}, Full}
  ],
  {{b, 0.1}, δ, 1 - δ, Appearance → "Open"},
  {{λ, lambdainit}, 0, 1, Appearance → "Open"},
  {xmax, xmaxinit},
  {{rangex, xmax}, δ, xmax, Appearance → "Open"},
  Dynamic[Style["x_max = " <> TextString[xmax]]]
]
```



We can apply the same idea to h -function version 1:

Downward limiter function, version 2b

$$f(x; b, c) = \frac{1}{\exp((\log(b)(cx - 1)))} + (ecb \log(b) + 1)x, \text{ for } x < -\frac{1}{c \log(b)} \equiv a$$

$$g(x; b, c) = \begin{cases} f(x; b, c) & \text{for } x < a \equiv eb \\ x & \text{for } x > a \equiv eb \end{cases}$$

$$h(x; b, \lambda) = \lambda g(x) + (1 - \lambda) \max(x, b), \quad 0 \leq \lambda \leq 1$$

$$f'(x) = d - c \log(b) b^{1-cx}$$

$$d = ecb \log(b) + 1$$

Constraints

Same as version 1.

Plot

```
In[690]:=  $\delta = 1 \star^{-8};$   
 $\text{bmax} = 1;$ 
```

```

cmax = 10;
binit = 0.1;
cinit = 0.1;
kinit = N[E + 2];
lambdainit = 0.5;
(*xtreme=E+2;*) (*The higher the max x, the smaller the range of c*)

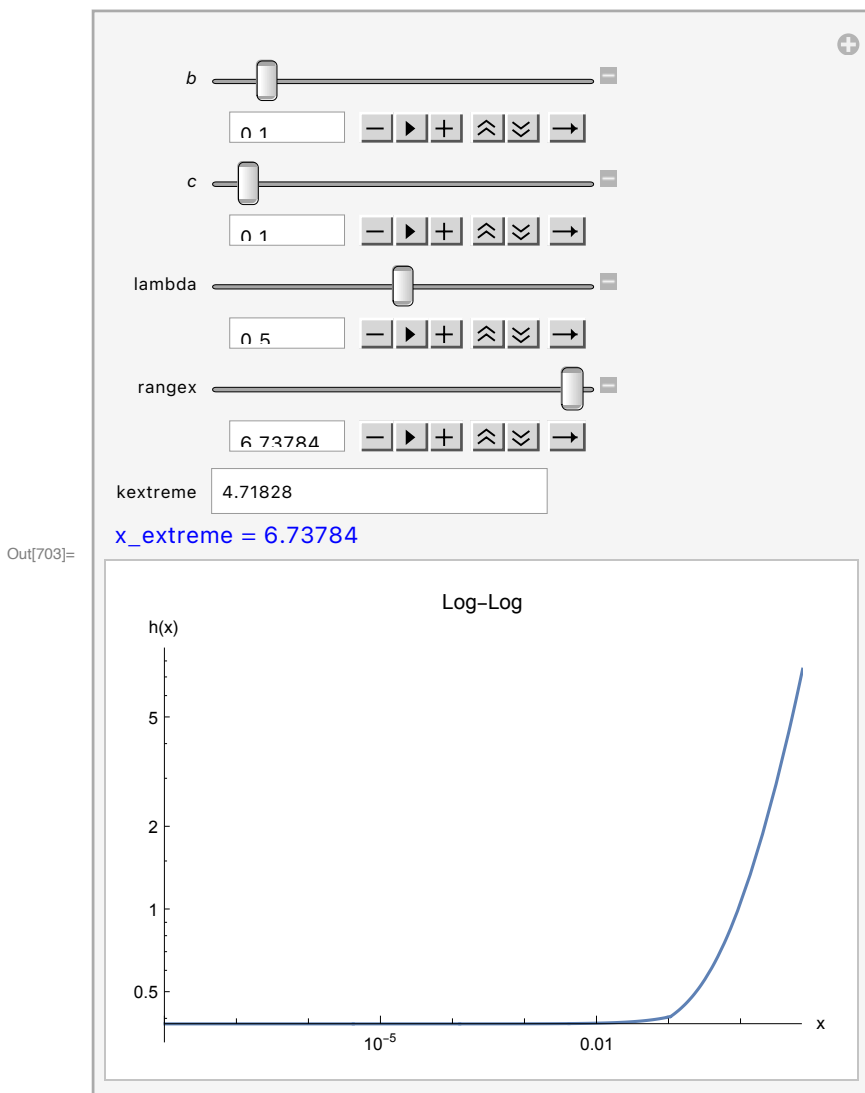
Clear[x, a, b, c]

(*aval[b_,c_,d_]:=1/c-log (d/(c log (b)))/(c log (b))*
(*aval[x_,b_,c_]:=
  Solve[Log[b]*(1-c*x)-Log[x]==Log[-E*b*c Log[b]],x][[1]][[2]]*)
aval[b_, γ_] := -1 / (γ Log[β]);
(*tauzero[b_, γ_] := γ * b*Log[b]*E;*)
(*tauinfn[xtreme_, b_, γ_] := γ * b*Log[b]*(E - b^(-γ*xtreme))*
tauinfn[kextreme_, β_, γ_] := γ * β * Log[β] * (E - kextreme)

f[x_, β_, γ_] := 1 / (Exp[Log[β] * (γ * x - 1)]) + (E * γ * β * Log[β] + 1) * x;
g[x_, β_, γ_] := Piecewise[{{f[x, β, γ], x < aval[β, γ]}, {g[x], x ≥ aval[β, γ]}}];
h[x_, β_, γ_, λ_] := λ * f[x, β, γ] + (1 - λ) * Max[x, β];

Manipulate[
  LogLogPlot[h[x, b, c, lambda],
    {x, δ, rangex},
    AxesLabel → {"x", "h(x)"},
    PlotLabel → "Log-Log",
    PlotRange → {{δ, rangex}, Full}
  ],
  {{b, binit}, δ, bmax - δ, Appearance → "Open"},
  (*{{c, 0.1}, δ, If[b>1, 1/(b *Log[b]* E), cmax],Appearance→"Open"}*)
  (*{{c, 0.1}, δ,If[b>1,NSolve[tauzero[b, γ]==1,γ,Reals][[1]][[2]],
    NSolve[tauinfn[b, γ]==1,γ,Reals][[1]][[2]],Appearance→"Open"}*)
  (*{{c, 0.1}, δ, Min[1/(b*Log[b]*(E-xtreme)), cmax],Appearance→"Open"},*)
  {{c, cinit}, δ,
    Min[NSolve[tauinfn[kextreme, b, γ] == 1, γ, PositiveReals][[1]][[2]], cmax],
    Appearance → "Open"},
  {{lambda, lambdainit}, 0, 1, Appearance → "Open"},
  (*{{rangex, xtreme}, 2*δ, xtreme, Appearance→"Open"},*)
  {{rangex, -Log[kinit] / (cinit * Log[binit])},
    2 * δ, -Log[kextreme] / (c * Log[b]), Appearance → "Open"},
  (*{xtreme, N[E+2]}*)
  {kextreme, kinit},
  Dynamic[Style["x_extreme = " <> TextString[N[-Log[kextreme] / (c * Log[b])]],
    12, If[kextreme > E, Blue, Red]]]
  (*{{c,1/(2*Log[b]*(b-1))},δ, Min[maxc,1/(Log[b]*(b-1))-δ],
    Appearance→"Open"}*)
]

```



Version 2b offers more control, but is also much more difficult to calibrate and can be dangerously misspecified if not implemented with great care.

Downward limiter function, version 3

$$f(x; a, c) = a \left(x - b + \frac{1}{4a} \right)^2 + b$$

$$h(x; a, b) = \begin{cases} b & , \text{ for } 0 < x \leq b - \frac{1}{4a} \\ f(x; a) & , \text{ for } b - \frac{1}{4a} < x \leq b + \frac{1}{4a} \\ x & , \text{ for } x < b + \frac{1}{4a} \end{cases}$$

```

In[704]:= ainit = 0.2;
binit = 2;
amax = 1;
bmax = 10;

 $\delta = 10^{-8}$ ;

Clear[x, a, b, c]

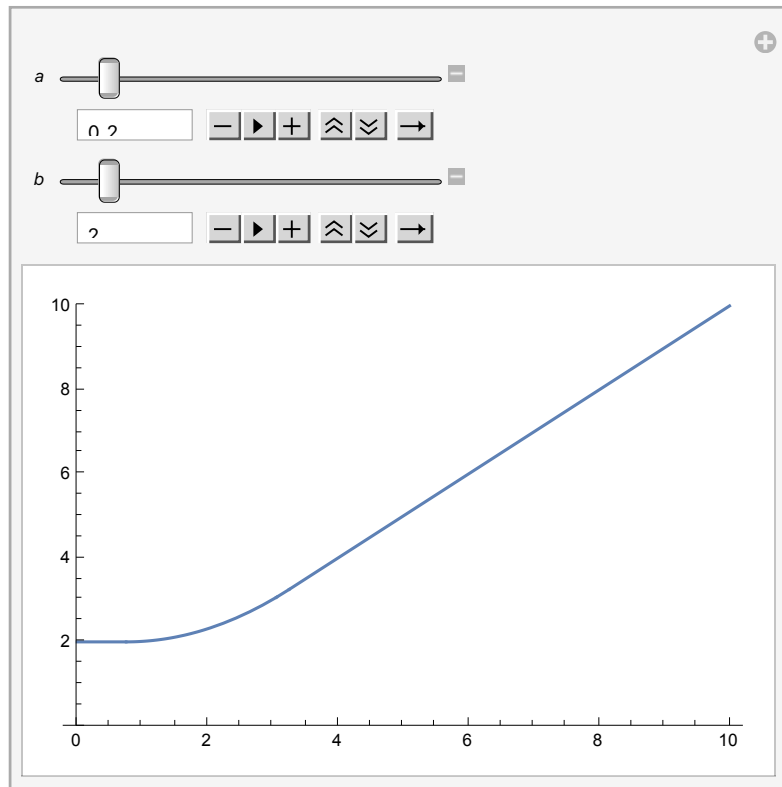
f[x_, a_, b_] := a * (x - b + 1 / (4 * a)) ^ 2 + b;

h[x_, a_, b_] := Piecewise[
{
  {b, x < b - 1 / (4 * a)},
  {f[x, a, b], b - 1 / (4 * a) ≤ x && x ≤ b + 1 / (4 * a)},
  {x, x > b + 1 / (4 * a)}
}
]

Manipulate[
  Plot[
    h[x, a, b],
    {x,  $\delta$ , bmax},
    PlotRange → {0, 10}
  ],
  {{a, ainit}, 1 / (4 * b), amax, Appearance → "Open"},
  {{b, binit}, 1 / (4 * a), bmax, Appearance → "Open"}
]

```


Out[712]=



Constraints

The constraints on the limiting multiplier are:

Constraints on x

$x \geq 0$ (because x is a standard deviation)

Constraints on a

Minimum value of a :

$$b > \frac{1}{4a} \Leftrightarrow a > \frac{1}{4b}$$

Constraints on b

$b > \frac{1}{4a}$ because otherwise the point where f touches $y = b$ will be below $x = 0$, and $f(x)$ will overshoot the limit of $f(0) = b$.

Soft knee upward limiter function

Soft knee upward limiter function, version 1

$$f(x; b, c) = b - \frac{b - \frac{bc-1}{c}}{\exp\left(c\left(x - \frac{bc-1}{c}\right)\right)}$$

$$f'(x; b, c) = c\left(b - \frac{bc-1}{c}\right) \exp\left(c\left(\frac{bc-1}{c} - x\right)\right)$$

$$h(x; b, c) = \begin{cases} f(x; b, c) & \text{for } x > \frac{bc-1}{c} \\ x & \text{for } 0 < x \leq \frac{bc-1}{c} \end{cases}$$

Constraints

The constraints on the limiting multiplier are:

Constraints on x

$x \geq 0$ (because x is a standard deviation)

Constraints on b

$b > 0$ because we take log of b (because b is a risk value, i.e. a standard deviation).

Given c , then $b > \frac{1}{c}$.

Constraints on c

$c > \frac{1}{b}$

Plot

Constraint on x :

$x > 0$

Constraint on b :

$b > 0$

Constraint on c :

$c > \frac{1}{b}$

```

In[713]:= Clear[b, c, x]

bmax = 10;
cmax = 20;

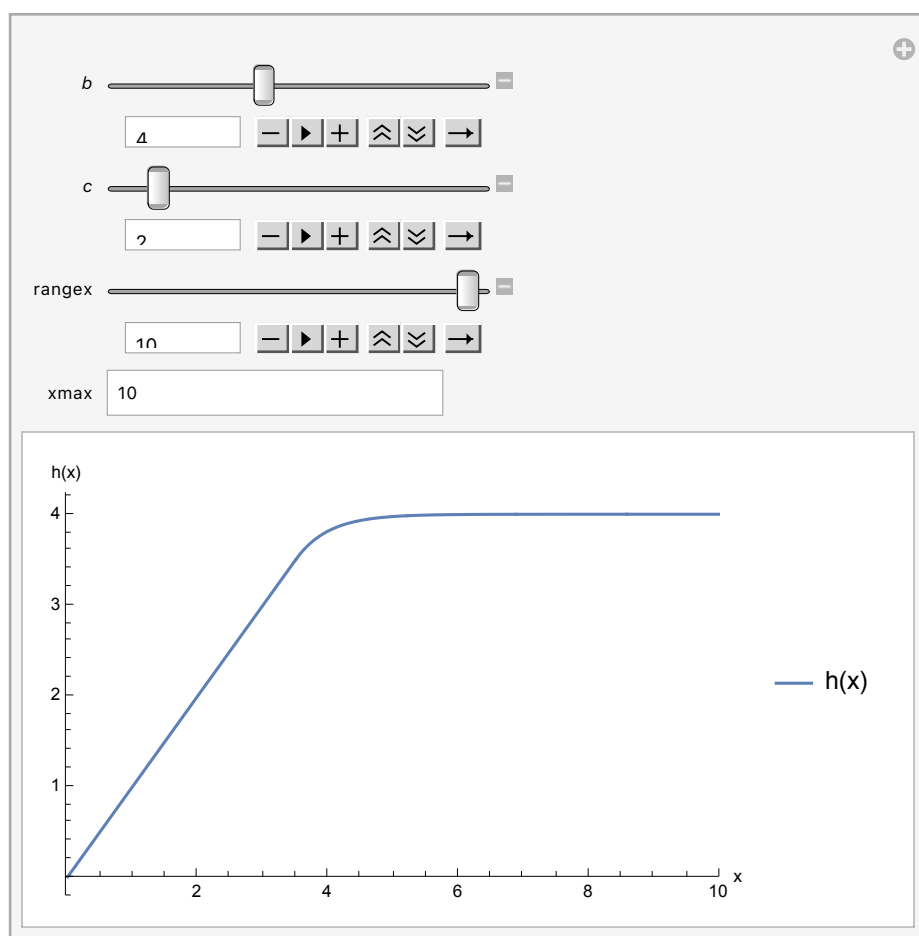
δ = 1*^-12;

fup[x_, b_, c_] := b -  $\left(b - \left(\frac{b * c - 1}{c}\right)\right) / \text{Exp}\left[c \left(x - \left(\frac{b * c - 1}{c}\right)\right)\right]$ 
gup[x_] := x
hup[x_, b_, c_] :=
  Piecewise[{{fup[x, b, c], x > (b * c - 1) / c}, {gup[x], x ≤ (b * c - 1) / c}}]

Manipulate[
  Plot[
    hup[x, b, c],
    {x, δ, rangex},
    PlotLegends → {"h(x)"},
    AxesLabel → {"x", "h(x)"},
    PlotRange → {{δ, rangex}, Full}
  ],
  {{b, 4}, δ, bmax, Appearance → "Open"},
  {{c, 2}, 1 / b + δ, cmax, Appearance → "Open"},
  {{rangex, 10}, 2 * δ, xmax, Appearance → "Open"},
  {xmax, 10}
]

```

Out[720]=



Soft knee upward limiter function, version 2

$$f(x; b, c) = b - \frac{b - \frac{bc-1}{c}}{\exp\left(c\left(x - \frac{bc-1}{c}\right)\right)}$$

$$g(x; b, c) = \begin{cases} f(x; b, c) & \text{for } x > \frac{bc-1}{c} \\ x & \text{for } x \leq \frac{bc-1}{c} \end{cases}$$

$$h(x; b, c, \lambda) = \lambda g(x; b, c) + (1 - \lambda) \min(x, b), \quad 0 \leq \lambda \leq 1$$

Constraints

Plot

```
In[721]:= Clear[b, c, x]

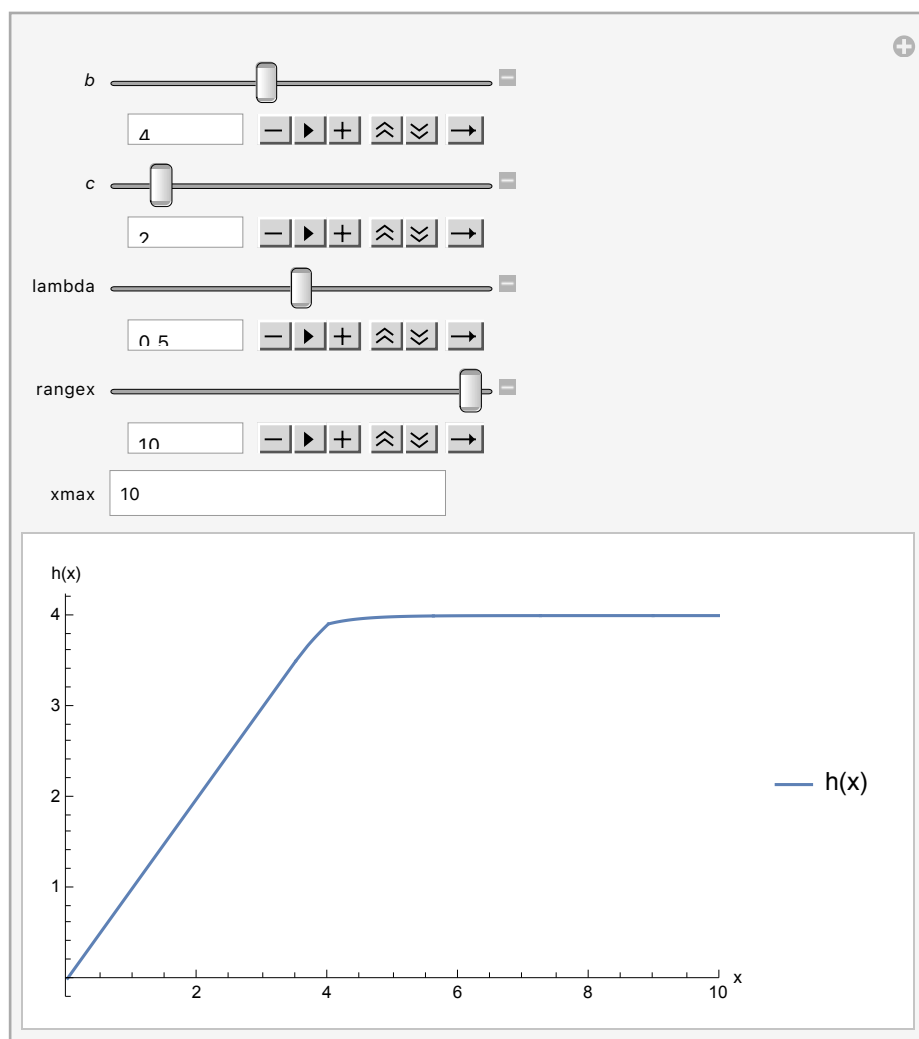
bmax = 10;
cmax = 20;
lambdainit = 0.5;

δ = 1*^-12;
fup[x_, b_, c_] := b -  $\left(b - \left(\frac{b * c - 1}{c}\right)\right) / \text{Exp}\left[c \left(x - \left(\frac{b * c - 1}{c}\right)\right)\right]$ 
gup[x_, b_, c_] :=
  Piecewise[{{fup[x, b, c], x > (b * c - 1) / c}, {x, x ≤ (b * c - 1) / c}}]

hup[x_, b_, c_, λ_] := λ * gup[x, b, c] + (1 - λ) * Min[x, b]

Manipulate[
  Plot[
    hup[x, b, c, lambda],
    {x, δ, rangex},
    PlotLegends → {"h(x)"},
    AxesLabel → {"x", "h(x)"},
    PlotRange → {{δ, rangex}, Full}
  ],
  {{b, 4}, δ, bmax, Appearance → "Open"},
  {{c, 2}, 1 / b + δ, cmax, Appearance → "Open"},
  {{lambda, lambdainit}, 0, 1, Appearance → "Open"},
  {{rangex, 10}, 2 * δ, xmax, Appearance → "Open"},
  {xmax, 10}
]
```

Out[729]=



Upward limiter function, version 3

$$f(x; a, b) = -a \left(x - b - \frac{1}{4a} \right)^2 + b$$

$$h(x; a, b) = \begin{cases} x & , \text{ for } 0 < x \leq b - \frac{1}{4a} \\ f(x; a) & , \text{ for } b - \frac{1}{4a} < x \leq b + \frac{1}{4a} \\ b & , \text{ for } x < b + \frac{1}{4a} \end{cases}$$

```

In[730]:= ainit = 0.2;
binit = 2;
amax = 1;
bmax = 10;

 $\delta = 10^{-8}$ ;

Clear[x, a, b]

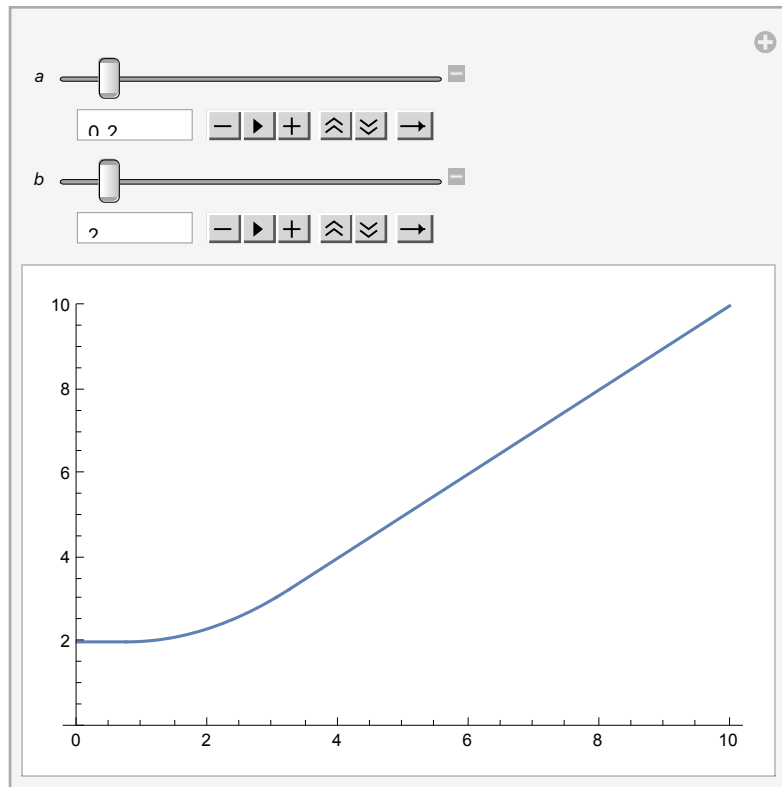
f[x_, a_, b_] := -a * (x - b - 1 / (4 * a)) ^ 2 + b;

h[x_, a_, b_] := Piecewise[
{
{x, x < b - 1 / (4 * a)},
{f[x, a, b], b - 1 / (4 * a) ≤ x && x ≤ b + 1 / (4 * a)},
{b, x > b + 1 / (4 * a)}
}
]

Manipulate[
Plot[
h[x, a, b],
{x,  $\delta$ , bmax},
PlotRange → {0, 10}
],
{{a, ainit}, 1 / (4 * b), amax, Appearance → "Open"},
{{b, binit}, 1 / (4 * a), bmax, Appearance → "Open"}
]

```

Out[738]=



Constraints

The constraints on the limiting multiplier are:

Constraints on x

$x \geq 0$ (because x is a standard deviation)

Constraints on a

Minimum value of a :

$$b > \frac{1}{4a} \Leftrightarrow a > \frac{1}{4b}$$

Constraints on b

$b > \frac{1}{4a}$ because otherwise the point where f touches $y = b$ will be below $x = 0$, and $f(x)$ will overshoot the limit of $f(0) = b$.

Derivation

Derivation of the downward limiter function, version 1

See derivation of the upward limiter function below.

We want to limit risk (standard deviation) downward.

Below $x = a$, limit y with a function, f , such that y approaches a limit, b , as x approaches 0 from

above.

Above $x = a$, we want the input x to pass unchanged to the output y , i.e. $g(x) = x$.

So we want to construct a piecewise function which is equal to $f(x)$ below $x = a$ and equal to $g(x)$ above $x = a$.

Initial idea for f :

$f(x)$ should tend to the limit b , as x tends to 0 from above.

$f'(x)$ should tend to 0, as x tends to 0 from above.

This gives us the following idea:

$$f(x; a, b) = \frac{a}{e^{\log(b)(x-a)}}$$

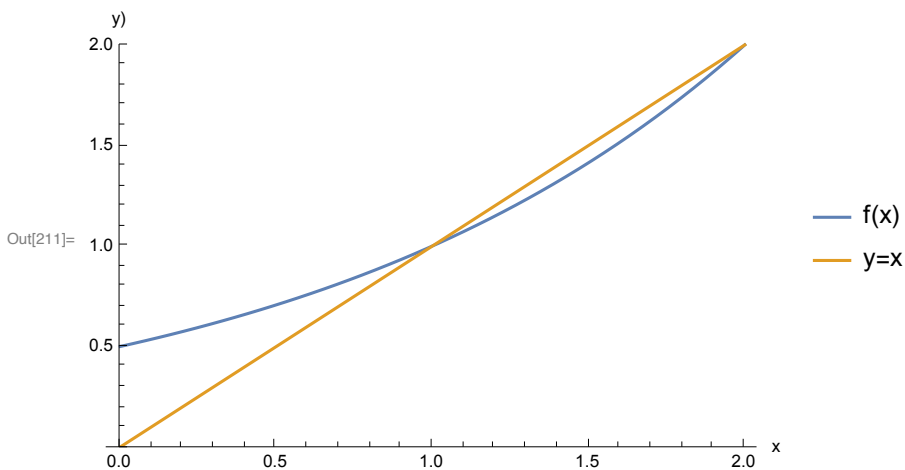
$$f'(x; a, b) = -a \log(b) b^{a-x}$$

Since $g(x) = x$, it must be true that in the point where $f(x)$ meets $g(x)$, $f(a) = a$.

```
In[210]:= f1[x_, a_, b_] := a / (Exp[Log[b] * (x - a)])
```

Let's set the limit to $b = 0.5$ and let f meet g in $(1, 1)$.

```
In[211]:= Plot[
  {f1[x, 1, 0.5], x}, {x, 0, 2}, PlotRange -> {0, 2},
  PlotLegends -> {"f(x)", "y=x"},
  AxesLabel -> {"x", "y"}
]
```

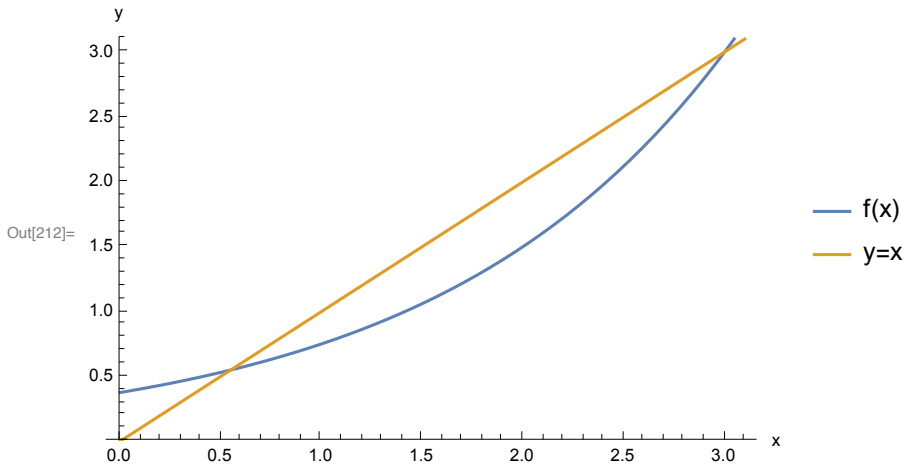


f does meet g in $(1, 1)$, but the combined curve will not be smooth.

And it won't work for other values of a than 1!

Now, set $a = 3$:

```
In[212]:= Plot[
  {f1[x, 3, 0.5], x}, {x, 0, 3.1},
  PlotRange -> {0, 3.1},
  PlotLegends -> {"f(x)", "y=x"},
  AxesLabel -> {"x", "y"}
]
```



The blue line does not hit b at $x = 0$:

$$f(x=0; a, b) = \frac{a}{e^{\log(b)(0-a)}} = ab^a$$

So $f(x)$ will only approach b from above when $a = 1$.

We arrive at the soft knee limiter function

$$f(x) = \frac{1}{\exp(\log(b)(c x - 1))} + d x$$

with

$$f'(x) = d - c \log(b) b^{1-cx}$$

We need to be able to set a fixed limit, b . Then by adjusting the “softness” of the knee and the horizontal “stretch” of the curve, it should be possible to find a point where g tangents f .

The x -value which we earlier called a , i.e. the x -value at which $f(x)$ and $g(x)$ meet, is now given implicitly by $f(x)$. We will derive a below.

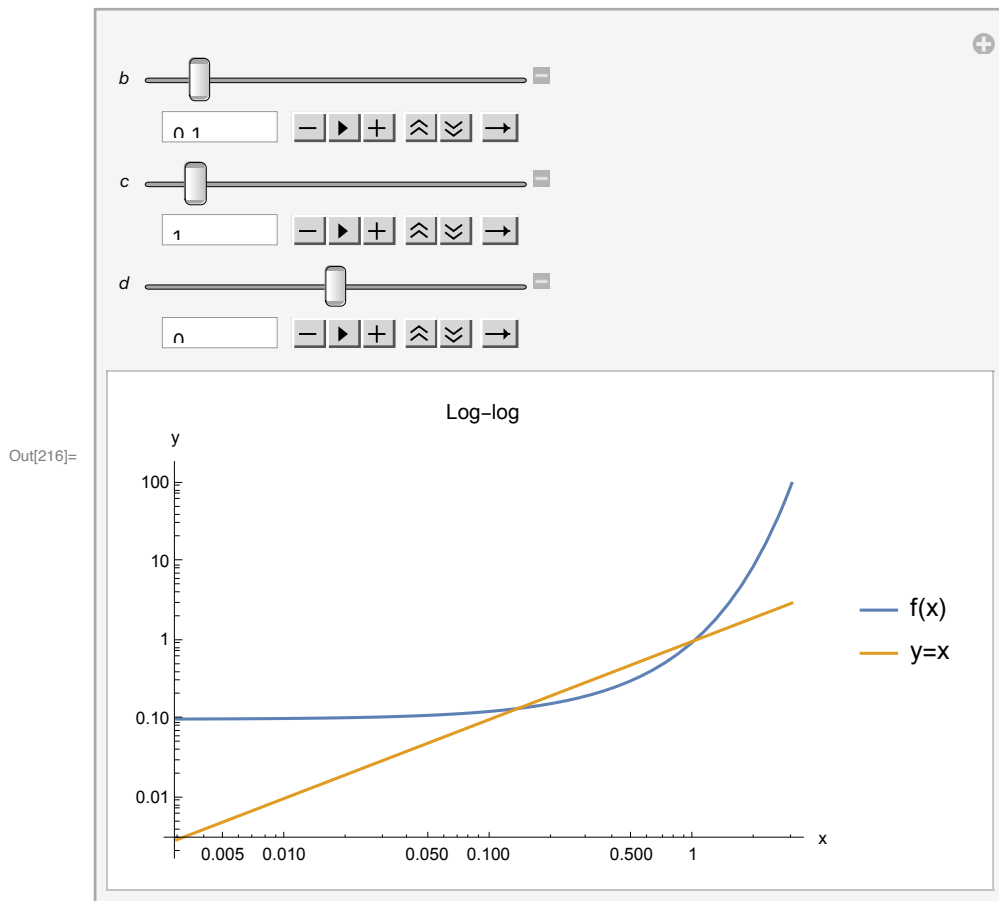
In this plot, c will stretch the blue curve, while d adjusts the softness of the knee:

```

In[213]:= Clear[x, b, c, d]
f6temp[x_, b_, c_, d_] := 1 / (Exp[Log[b] * (c * x - 1)]) + d * x
df6[x_] = D[f6temp[x, b, c, d], x]
Manipulate[
  LogLogPlot[
    {f6temp[x, b, c, d], x},
    {x, 0, 3},
    PlotLegends -> {"f(x)", "y=x"},
    AxesLabel -> {"x", "y"},
    PlotLabel -> "Log-log"
  ],
  {{b, 0.1}, 0, 1, Appearance -> "Open"},
  {{c, 1}, 0.1, 10, Appearance -> "Open"},
  {{d, 0}, -1, 1, Appearance -> "Open"}
]

```

Out[215]= $d - c \log(b) b^{1-cx}$



$$f(x) = \frac{1}{\exp(\log(b)(cx-1))} + dx$$

$$f'(x) = d - c \log(b) b^{1-cx}$$

```
In[217]:= Clear[x, b, c, d]
f[x_, b_, c_, d_] := 1 / (Exp[Log[b] * (c * x - 1)]) + d * x
df[x_] = D[f[x, b, c, d], x]
```

```
Out[219]= d - c log(b) b^{1-c x}
```

Find x for which $f'(x; b, c, d) = 1$.

$$d - c \log(b) b^{1-cx} = 1$$

$$\Leftrightarrow c \log(b) b^{1-cx} = d - 1$$

$$\Leftrightarrow b^{1-cx} = \frac{d-1}{c \log(b)}$$

$$\Leftrightarrow (1 - cx) \log(b) = \log\left(\frac{d-1}{c \log(b)}\right)$$

$$\Leftrightarrow 1 - cx = \frac{\log\left(\frac{d-1}{c \log(b)}\right)}{\log(b)}$$

$$\Leftrightarrow x = \frac{1}{c} - \frac{\log\left(\frac{d-1}{c \log(b)}\right)}{c \log(b)}$$

Find a for which $f(x = a; b, c, d) = a$.

$$\frac{1}{\exp(\log(b)(cx-1))} + dx = x$$

$$\Leftrightarrow \frac{1}{\exp(\log(b)(cx-1))} = (1-d)x$$

$$\Leftrightarrow \exp(\log(b)(1-cx)) = (1-d)x$$

$$\Leftrightarrow \frac{\exp(\log(b)(1-cx))}{1-d} = x$$

$$\Leftrightarrow \log(b)(1-cx) - \log(1-d) = \log(x)$$

$$\Leftrightarrow \log(b)(1-cx) - \log(x) = \log(1-d)$$

```
In[220]:= Clear[x, b, c, d]
aval[x_, b_, c_, d_] := Solve[Log[b] * (1 - c * x) - Log[x] == Log[1 - d], x][[1]][[2]];
aval[x, b, c, d]
```

 **Solve:** Inverse functions are being used by Solve, so some solutions may not be found; use Reduce for complete solution information.

```
Out[222]=
```

$$\frac{W\left(-\frac{bc \log(b)}{d-1}\right)}{c \log(b)}$$

$$a = \frac{W\left(-\frac{bc \log(b)}{d-1}\right)}{c \log(b)}$$

Set $x = a$ and solve for d .

$$x = a$$

$$\Leftrightarrow \frac{1}{c} - \frac{\log\left(\frac{d-1}{c \log(b)}\right)}{c \log(b)} = \frac{W\left(-\frac{b c \log(b)}{d-1}\right)}{c \log(b)}$$

$$\Leftrightarrow \log(b) - \log\left(\frac{d-1}{c \log(b)}\right) = W\left(-\frac{b c \log(b)}{d-1}\right)$$

$$\Leftrightarrow d = e^{b c \log(b) + 1}$$

In[223]:= `Solve[Log[b] - Log[(d - 1) / (c * Log[b])] == a, d] * c * Log[b], d][[1]][[2]]`

 **Solve:** Inverse functions are being used by Solve, so some solutions may not be found; use Reduce for complete solution information.

Out[223]= $e^{b c \log(b) + 1}$

Plug the expression of d into the expression of a :

$$a = -\frac{1}{c \log(b)}$$

In[224]:= `Clear[x, b, c]`
`aval[x_, b_, c_] :=`
`Solve[Log[b] * (1 - c * x) - Log[x] == Log[-E * b * c * Log[b]], x][[1]][[2]];`
`aval[`
`x,`
`b,`
`c]`

 **Solve:** Inverse functions are being used by Solve, so some solutions may not be found; use Reduce for complete solution information.

Out[226]= $-\frac{1}{c \log(b)}$

We can now plot the final version of h ,
 where

$$h(x; b, c) = \begin{cases} f(x; b, c) = \frac{1}{\exp((\log(b)(c x - 1)))} + (e^{b c \log(b) + 1}) x & , \text{ for } x < -\frac{1}{c \log(b)} \\ g(x) = x & , \text{ for } x \geq -\frac{1}{c \log(b)} \end{cases}$$

In[227]:= `tauinf[b_, c_] := c * b * Log[b] * (E - xtreme)`

In[228]:= `NSolve[tauinf[b, c] == 1, c, Reals]`

Out[228]= $\left\{ \left\{ c \rightarrow -\frac{1.}{b \text{ xtreme} \log(b) - 2.71828 b \log(b)} \text{ if } \text{condition} \right\} \right\}$

```

In[739]:=  $\delta = 1 \cdot 10^{-12}$ ;
maxc = 10; (*Chosen maximum value of c
(not theoretical max, which can be Inf for b=1).*)
maxx = E + 2; (*The higher the max x, the smaller the range of c*)

Clear[x, a, b, c]

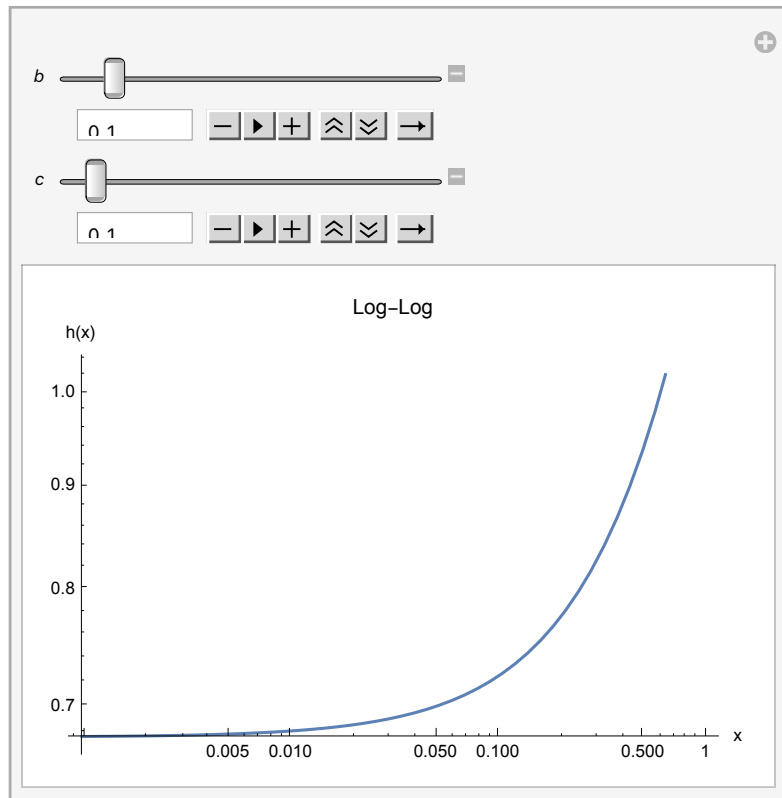
(*aval[b_,c_,d_]:=1/c-log (d/(c log (b)))/(c log (b))*
(*aval[x_,b_,c_]:=
Solve[Log[b]*(1-c*x)-Log[x]==Log[-E*b*c Log[b]],x][[1]][[2]];*)
aval[b_, c_] := -1 / (c Log[b]);
tauzero[b_,  $\gamma$ ] :=  $\gamma * b * \text{Log}[b] * E$ ;
tauinf[b_,  $\gamma$ ] :=  $\gamma * b * \text{Log}[b] * (E - \text{maxx})$ ; (*maxx =  $b^{(-c \cdot \text{xtreme})}$ *)

f[x_, b_, c_] := 1 / (Exp[Log[b] * (c * x - 1)] + (E * b * c Log[b] + 1) * x);
g[x_] := x;
h[x_, b_, c_] :=
Piecewise[{{f[x, b, c], x < aval[b, c]}, {g[x], x  $\geq$  aval[b, c]}}];

Manipulate[
LogLogPlot[h[x, b, c],
{x, 0, 1 -  $\delta$ },
AxesLabel -> {"x", "h(x)"},
PlotLabel -> "Log-Log",
{{b, 0.1},  $\delta$ , 0.999, Appearance -> "Open"},
(*{{c, 0.1},  $\delta$ , If[b>1, 1/(b * Log[b] * E), cmax], Appearance -> "Open"}*)
{{c, 0.1},  $\delta$ , If[b > 1, NSolve[tauzero[b,  $\gamma$ ] == 1,  $\gamma$ , Reals][[1]][[2]],
NSolve[tauinf[b,  $\gamma$ ] == 1,  $\gamma$ , Reals][[1]][[2]], Appearance -> "Open"}
(*{{c, 1/(2*Log[b]*(b-1))},  $\delta$ , Min[maxc, 1/(Log[b]*(b-1)) -  $\delta$ ],
Appearance -> "Open"}*)
]

```

Out[749]=



Constraints

Finally, we need to calculate the valid parameter ranges for b and c , given $x \geq 0$. These will be ones where f' is always positive for positive values of x .

So, we have $f'(x) = d - c \log(b) b^{1-cx}$ and $d = e b c \log(b) + 1$, and we need:

$$d - c \log(b) b^{1-cx} = e b c \log(b) + 1 - c \log(b) b^{1-cx} = c b \log(b) (e - b^{-cx}) + 1 > 0$$

$$\Leftrightarrow c b \log(b) (e - b^{-cx}) < 1$$

Let

$$\tau(x; b, c) := c b \log(b) (e - b^{-cx})$$

$b > 1$

If $b > 1$, then $a = -\frac{1}{c \log(b)} < 0$, which means that $h(x) = g(x) \forall x > 0$. The softknee function will never kick in. So in implementation we should constrain b to $0 < b < 1$.

Nevertheless, let's examine the case $b > 1$.

```

In[750]:= b = 1.2;
(*m=1;
n=2*m+2;*)

Clear[c, x]

tauvals[x_, c_, b_] := c * b * Log[b] * (E - b^(-c x))

(*xvalues=N[Flatten[Append[{0}, 10^(Range[-m,m])]]];*)

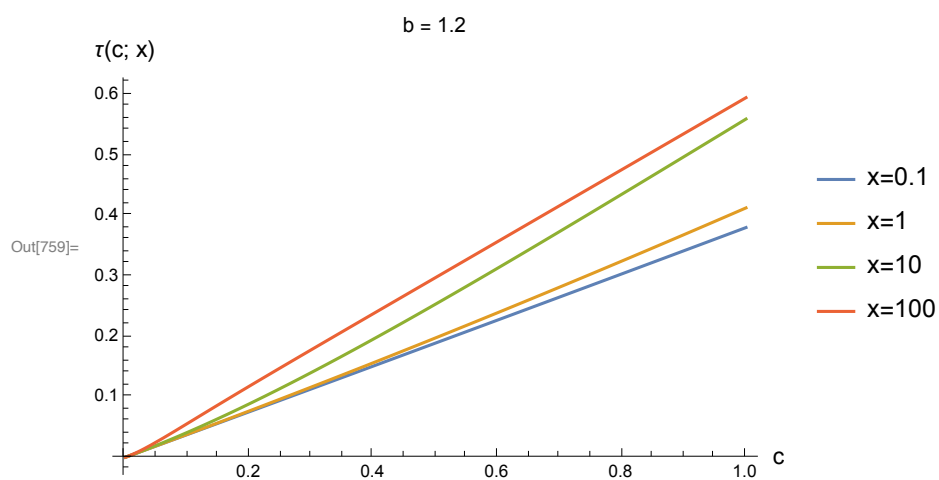
xvalues = {0.1, 1, 10, 100};
n = Length[xvalues];

legends = {};
For[
  i = 1, i < n + 1, i++,
  legends = Append[legends, StringJoin[{"x=", ToString[xvalues[[i]]}]]]
]

out = {};
For[
  i = 1, i < n + 1, i++, out = Append[out, tauvals[xvalues[[i]], c, b]]
]

Plot[
  out,
  {c, 0, 1},
  AxesLabel -> {Style["c", 12], Style[" $\tau(c; x)$ ", 12]},
  PlotLegends -> legends,
  PlotLabel -> StringJoin["b = ", ToString[b]]
]

```




```

In[250]:= b = 1.2;
(*m=1;
n=2*m+2;*)

Clear[c, x]

tauvals[x_, c_, b_] := c * b * Log[b] * (E - b^(-c x))

(*xvalues=N[Flatten[Append[{0}, 10^(Range[-m,m])]]];*)

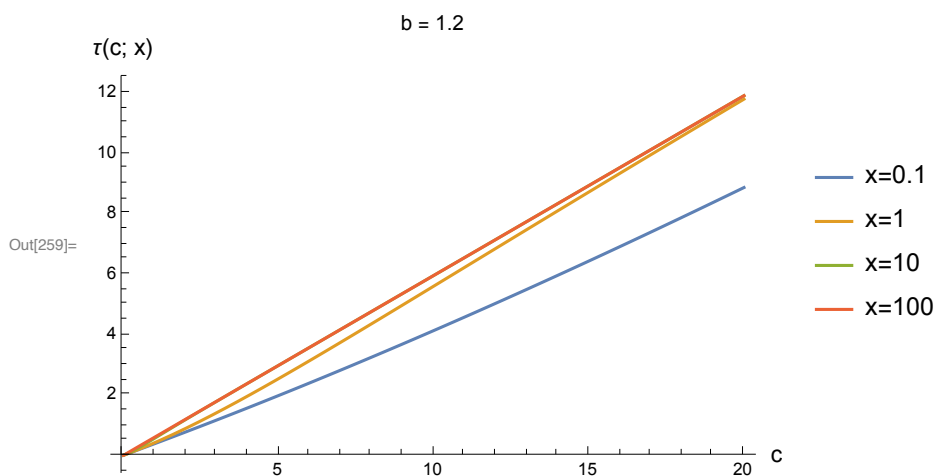
xvalues = {0.1, 1, 10, 100};
n = Length[xvalues];

legends = {};
For[
  i = 1, i < n + 1, i++,
  legends = Append[legends, StringJoin[{"x=", ToString[xvalues[[i]]}]]]
]

out = {};
For[
  i = 1, i < n + 1, i++, out = Append[out, tauvals[xvalues[[i]], c, b]]
]

Plot[
  out,
  {c, 0, 20},
  AxesLabel -> {Style["c", 12], Style[" $\tau(c; x)$ ", 12]},
  PlotLegends -> legends,
  PlotLabel -> StringJoin["b = ", ToString[b]]
]

```



```

In[260]:= b = 1.2;
(*m=1;
n=2*m+2;*)

Clear[c, x]

tauvals[x_, c_, b_] := c * b * Log[b] * (E - b^(-c x))

(*xvalues=N[Flatten[Append[{0}, 10^(Range[-m,m])]]];*)

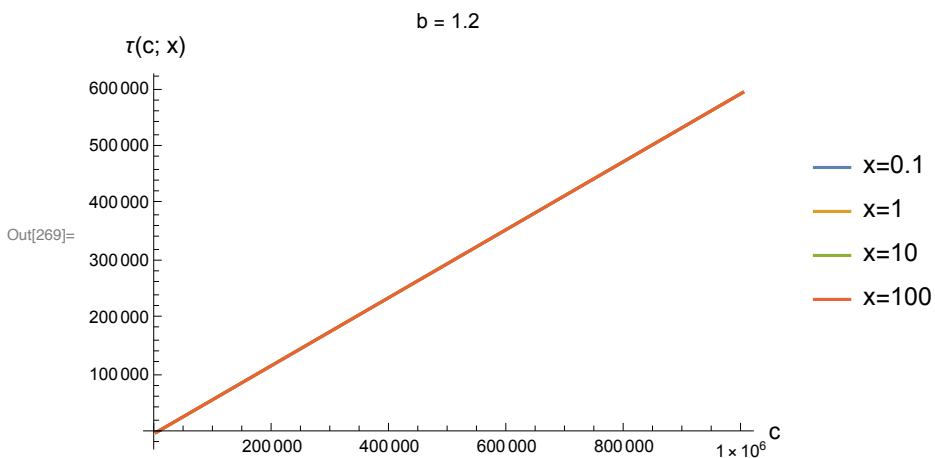
xvalues = {0.1, 1, 10, 100};
n = Length[xvalues];

legends = {};
For[
  i = 1, i < n + 1, i++,
  legends = Append[legends, StringJoin[{"x=", ToString[xvalues[[i]]}]]]
]

out = {};
For[
  i = 1, i < n + 1, i++, out = Append[out, tauvals[xvalues[[i]], c, b]]
]

Plot[
  out,
  {c, 0, 1000000},
  AxesLabel -> {Style["c", 12], Style[" $\tau(c; x)$ ", 12]},
  PlotLegends -> legends,
  PlotLabel -> StringJoin["b = ", ToString[b]]
]

```



$\tau(x; b, c)$ is monotonically increasing in c for all values of x , when $b > 1$.

Since we have an upper constraint on $\tau(x; b, c)$, the “worst case” is $x = \infty$. As the plot suggests, the

higher the value of x , the sooner $\tau(x; b, c)$ will reach the limit of 1, as c increases.

Look at the constraint again:

$$\tau(x; b, c) := c b \log(b) (e - b^{-cx}) < 1$$

For $b > 1$ and $c > 0$, $-b^{-cx} \rightarrow 0$ for $x \rightarrow \infty$, so $c b \log(b) (e - b^{-cx}) \rightarrow c b \log(b) e$.

In short:

$\tau(x; b, c) \rightarrow c b \log(b) e$ when $x \rightarrow \infty$ for all values of c .

$\tau(x; b, c)$ is increasing in c .

So the calculated max value of c will be the upper limit.

$b > 1, x = 0$

```
In[760]:= b = 1.2;
n = 1;

Clear[c, x]

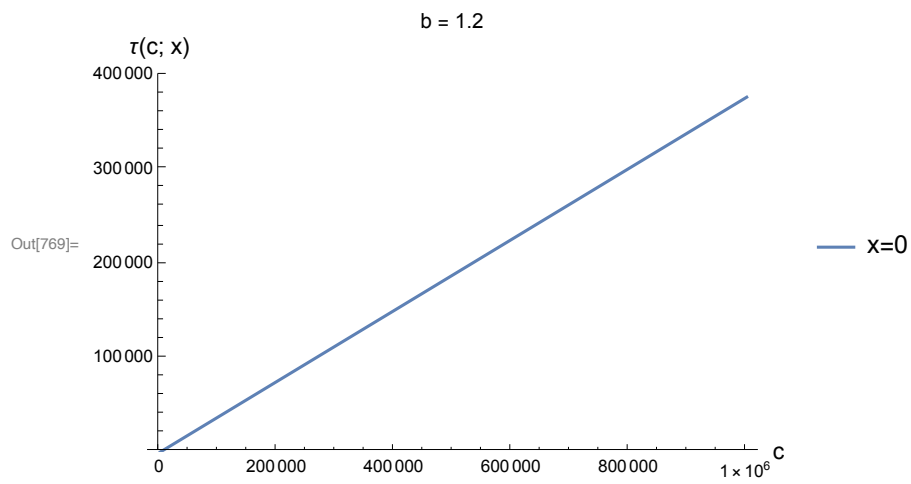
tauvals[x_, c_, b_] := c * b * Log[b] * (E - b^(-c x))

xvalues = {0};

legends = {};
For[
  i = 1, i < n + 1, i++,
  legends = Append[legends, StringJoin[{"x=", ToString[xvalues[[i]]}]]]
]

out = {};
For[
  i = 1, i < n + 1, i++, out = Append[out, tauvals[xvalues[[i]], c, b]]
]

Plot[
  out,
  {c, -1, 1000000},
  AxesLabel -> {Style["c", 12], Style["τ(c; x)", 12]},
  PlotLabel -> StringJoin["b = ", ToString[b]],
  PlotLegends -> legends,
  PlotRange -> {0, 400000}
]
```



For $b > 1$ and $x = 0$, $\tau(c; x)$ is increasing monotonically.

$$0 < b < 1$$

```

In[280]:= b = 0.2;
n = 10;

Clear[c, x]

tauvals[x_, c_, b_] := c * b * Log[b] * (E - b^(-c x))

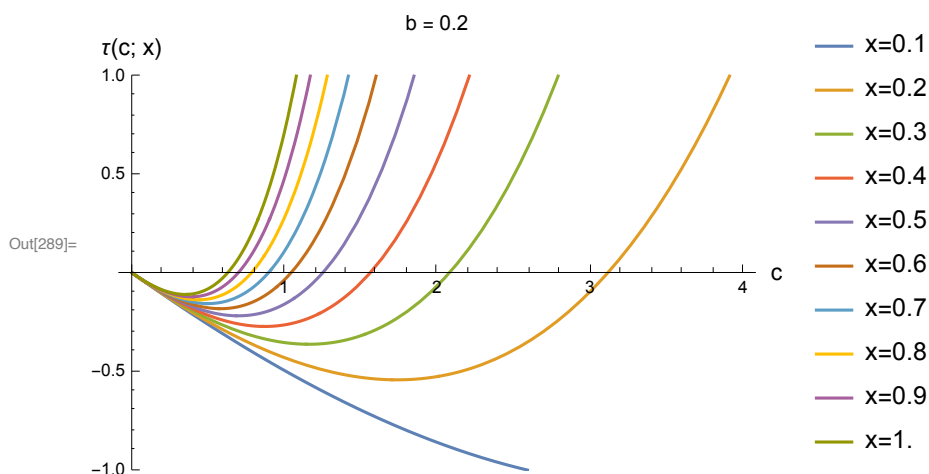
xvalues = N[Range[1, n] / 10];

legends = {};
For[
  i = 1, i < n + 1, i++,
  legends = Append[legends, StringJoin[{"x=", ToString[xvalues[[i]]}]]]
]

out = {};
For[
  i = 1, i < n + 1, i++, out = Append[out, tauvals[xvalues[[i]], c, b]]
]

Plot[
  out,
  {c, 0, 4},
  AxesLabel -> {Style["c", 12], Style["τ(c; x)", 12]},
  PlotLabel -> StringJoin["b = ", ToString[b]],
  PlotLegends -> legends,
  PlotRange -> {-1, 1}
]

```



$\tau(x; b, c)$ is not monotonically increasing in c .

So for which extreme value of x should we solve $\tau(x; b, c)$ for c ? 0 or ∞ ?

For positive values of $\tau(x; b, c)$, $\tau(x; b, c)$ is increasing in c , and $\tau(x; b, c)$ eventually exceeds 1 for all values of x , if we increase c enough.

Since we have an upper constraint on $\tau(x; b, c)$, the “worst case” is $x = \infty$. As the plot suggests, the higher the value of x , the sooner $\tau(x; b, c)$ will reach the limit of 1, as c increases.

Look at the constraint again:

$$\tau(x; b, c) := c b \log(b) (\theta - b^{-cx}) < 1$$

For $0 < b < 1$ and $c > 0$, $-b^{-cx} \rightarrow -\infty$ for $x \rightarrow \infty$, and $c b \log(b) < 0$, so $c b \log(b) (\theta - b^{-cx}) \rightarrow \infty$.

In short:

$\tau(x; b, c) \rightarrow \infty$ when $x \rightarrow \infty$ for all values of c .

In the extreme case, where x is extremely high, no value of c will be valid.
So we need to determine a practical range of x . For this we need heuristics.

$$0 < b < 1, x = 0$$

```

In[290]:= b = 0.2;
n = 1;

Clear[c, x]

tauvals[x_, c_, b_] := c * b * Log[b] * (E - b^(-c x))

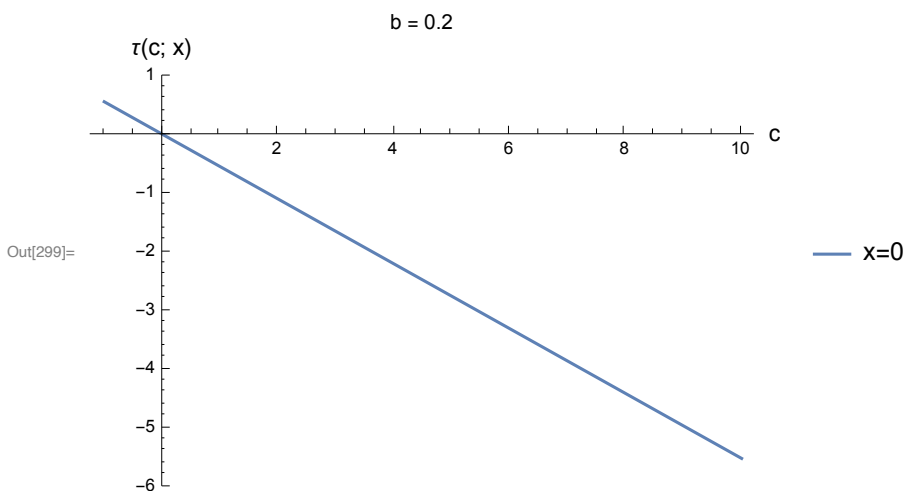
xvalues = {0};

legends = {};
For[
  i = 1, i < n + 1, i++,
  legends = Append[legends, StringJoin[{"x=", ToString[xvalues[[i]]}]]]
]

out = {};
For[
  i = 1, i < n + 1, i++, out = Append[out, tauvals[xvalues[[i]], c, b]]
]

Plot[
  out,
  {c, -1, 10},
  AxesLabel -> {Style["c", 12], Style[" $\tau(c; x)$ ", 12]},
  PlotLabel -> StringJoin["b = ", ToString[b]],
  PlotLegends -> legends,
  PlotRange -> {-6, 1}
]

```



For $0 < b < 1$ and $x = 0$, $\tau(c)$ is monotonically decreasing, and always negative for $c > 0$.

Implementation notes

For implementation, a way forward might be to solve $\tau(x; b, c) = 1$ for c given a value of b , to find the range of c where $\tau(x; b, c) < 1$.

$$\tau(x; b, c) := c b \log(b) (e - b^{-cx}) < 1$$

Given conditions $b > 0, x > 0$:

$$c b \log(b) (e - b^{-cx}) < 1$$

$$\Leftrightarrow b \log(b) (c e - c b^{-cx}) < 1$$

$$\Leftrightarrow (c e - c b^{-cx}) < \frac{1}{b \log(b)} \text{ for } b \log(b) > 0 \Leftrightarrow b > 1$$

$$\Leftrightarrow (c e - c b^{-cx}) > \frac{1}{b \log(b)} \text{ for } b \log(b) < 0 \Leftrightarrow 0 < b < 1$$

If $0 < b < 1$, c should be greater than the calculated threshold.

If $b < 1$, c should be less than the calculated threshold (and greater than 0).

We can't solve this inequality analytically:

In[300]:=

```
Clear[x, b, c]
b = 0.2;
Solve[c * E - c * b^(-c * x) == 1 / (b * Log[b]), c, Reals, Assumptions -> {x > 0, x < 1}]
```

Solve: Solve was unable to solve the system with inexact coefficients or the system obtained by direct rationalization of inexact numbers present in the system. Since many of the methods used by Solve require exact input, providing Solve with an exact version of the system may help.

Solve: Solve was unable to solve the system with inexact coefficients or the system obtained by direct rationalization of inexact numbers present in the system. Since many of the methods used by Solve require exact input, providing Solve with an exact version of the system may help.

Out[302]=

```
Solve[E c - c 0.2^(-c x) == -3.10667, c, R, Assumptions -> {x > 0, x < 1}]
```

In[303]:=

```
Clear[x, b, c]
```

In[304]:=

```
Solve[c * b * Log[b] * (E - b^(-c * x)) < 1, c, Reals, Assumptions -> x > 0]
```

Solve: This system cannot be solved with the methods available to Solve.

Out[304]=

```
Solve[b c log(b) (e - b^(-c x)) < 1, c, R, Assumptions -> x > 0]
```

In[305]:=

Solve[c*b*Log[b]*(E-b^(-c*x))<1, c, Reals, Assumptions->x>0]
(no interpretations available) ?

The strategy is to solve $(c e - c b^{-cx}) > \frac{1}{b \log(b)}$ for c with two extreme values of x .

$b > 1, x = 0$

First, we set x to 0:

$$\tau(x; b, c) := c b \log(b) (e - b^{-c x}) < 1$$

Let's define $\tau(c) := \tau(c; x, b) \big|_{x=\infty} = c b \log(b) (e - 1)$
for $c > 0$.

Constraint:

$$c < \frac{1}{b \log(b) (e-1)}$$

Here we are imposing the constraint $c > 0$, to simplify the problem.

```
In[306]:= Clear[x, b, c]
b = 1.2;
x = 0;

(*tauzero[b_, c_] := c * b*Log[b]*(E-1)*)

(*NSolve[c*E-c*b^(-c*x)==1/(b*Log[b]),c,Reals]*)
(*NSolve[tauzero[b, c]==1,c,Reals]*)

1 / ( b * Log[b] * (E - 1) )
```

Out[309]= 2.66003

So the constraint for $b > 1$ and $x = 0$ is:

$$c < 2.66$$

But we already know from above, that the relevant extreme value of x is ∞ .

So we can ignore this constraint.

$b > 1, x = \infty$

$$\tau(x; b, c) := c b \log(b) (e - b^{-c x}) < 1$$

Let's define $\tau(c) := \tau(c; x, b) \big|_{x=\infty} = c b \log(b) e$

Constraint:

$$c < \frac{1}{b \log(b) e}$$

```
In[310]:= tau2[b_, c_] := c * b * Log[b] * E
```

```
In[311]:= Clear[x, b, c]
b = 1.2;
x = 0;

(*tauinf[b_, c_] := c * b*Log[b]*E*)

(*NSolve[tauinf[b, c]==1,c,Reals]*)

1 / (b * Log[b] * E)
```

Out[314]= 1.68146

So the constraint when $b > 1$ and $x = \infty$ is:
 $c < 1.68$

$$0 < b < 1, x = 0$$

$$\tau(x; b, c) := c b \log(b) (e - b^{-c x}) < 1$$

Let's define $\tau(c) := \tau(c; x, b) |_{x=\infty} = c b \log(b) (e - 1)$

Constraint:

$$c < \frac{1}{b \log(b) (e-1)}$$

```
In[315]:= Clear[x, b, c]
b = 0.2;
x = 0;

(*tauzero[b_, c_] := c * b*Log[b]*(E-1)*)

(*NSolve[c*E-c*b^(-c*x)==1/(b*Log[b]),c,Reals]*)
(*NSolve[tauzero[b, c]==1,c,Reals]*)

1 / (b * Log[b] * (E - 1))
```

Out[318]= -1.80801

For $0 < b < 1$ and $x = 0$, $\tau(c)$ is monotonically decreasing, and always negative for $c > 0$.

So ignoring the negative solution, there are no other constraints than

$$c > 0$$

But again we already know from above, that the relevant extreme value of x is ∞ .

So we can ignore this constraint, which is redundant anyway.

$$0 < b < 1, x = \infty$$

$$\tau(x; b, c) := c b \log(b) (e - b^{-c x}) < 1$$

Let's define $\tau(c) := \tau(c; x, b) |_{x=\infty} = c b \log(b) (e - \infty)$

Constraint:

$$c < \frac{1}{b \log(b) (e - \infty)}$$

For implementation we replace the theoretical maximum possible x -value, ∞ , with x_{extreme} . In practice, this value is chosen heuristically. The trade-off is that the higher the x_{extreme} , the more narrow the valid range of c . For $x_{\text{extreme}} = \infty$ there will be no valid value of c . For values of x_{extreme} very close to e , the max value of c can get very big, and the resulting $h(x)$ may have negative slope for some values of x .

For b values close to 1, c can get very big.

For implementation it can be a good idea to impose a maximum c value. But this is a bit tricky, because the most useful value will depend on b .

We set x to a very high number which is effectly ∞ . We find that $x = 1\,000\,000$ is a reasonable extreme value:

```
In[319]:= Clear[x, b, c]
b = 0.2;
xtreme = 1 000 000;

(*tauinf[b_, c_] := c * b*Log[b]*(E - xtreme)*)

(*NSolve[c*E-c*b^(-c*x)==1/(b*Log[b]),c,Reals]*)

(*NSolve[tauinf[b, c]==1,c,Reals]*)

1 / ( b * Log[b] * (E - xtreme))
```

Out[322]= 3.10668×10^{-6}

$\tau(x; b, c) \rightarrow \infty$ when $x \rightarrow \infty$ for all values of c .

This means that the max value of c goes to 0 when $x \rightarrow \infty$, for $0 < b < 1$.

In the extreme case, where x is extremely high, no value of c will be valid.

So we need to determine a practical range of x . For this we need heuristics.

If the practical use case is x representing risk of daily returns as a standard deviation, a max x value of 1 should be extremely high. Let's multiply that by 100. Let's also pick a more realistic b value of 0.001.

```
In[323]:= Clear[x, b, c]
b = 0.001;
xtreme = 10;

(*tauinf[b_, c_] := c * b*Log[b]*(E - xtreme)*)

(*NSolve[c*E-c*b^(-c*x)==1/(b*Log[b]),c,Reals]*)

(*NSolve[tauinf[b, c]==1,c,Reals]*)
1 / (b * Log[b] * (E - xtreme))
Out[326]= 19.8806
```

This gives us a maximum c value of 19.88.

```
In[327]:= Clear[x, b, c]
b = 0.1;
xtreme = 10;

tauinf[b_, c_] := c * b * Log[b] * (E - b^(-c * xtreme))

cmax = NSolve[tauinf[b, c] == 1, c, Reals][[2]][[1]][[2]];
cmax
b^(-cmax * xtreme)
Out[332]= 0.150053
Out[333]= 31.6611
```

Derivation of the downward limiter function, version 2

Initial idea:

The h -function in version 1 offers a limited minimum “softness” of the knee. In other words, we can’t achieve a hard knee with that function.

To enable the limiter function to offer anything between a hard and a very soft knee, we may simply interpolate between $f(x)$ and $\max(b, x)$:

$$f(x) = \frac{1}{\exp(\log(b)(cx-1))}, \quad c = -\frac{1}{e b \log(b)}$$

$$g(x) = \begin{cases} f(x) & \text{for } x < a \\ x & \text{for } x > a \end{cases}$$

$$h(x) = \lambda g(x) + (1 - \lambda) \max(x, b), \quad 0 \leq \lambda \leq 1$$

This has considerable benefits, as it simplifies the f -function a lot.

$$f(x) = \frac{1}{\exp(\log(b)(cx-1))}$$

$$f'(x) = -c \log(b) b^{1-cx}$$

```
In[334]:= Clear[x, b, c]
f[x_, b_, c_] := 1 / (Exp[Log[b] * (c * x - 1)])
df[x_] = D[f[x, b, c], x]
```

```
Out[336]= c log(b) (-b^{1-cx})
```

Find x for which $f'(x; b, c) = 1$.

$$-c \log(b) b^{1-cx} = 1$$

$$\Leftrightarrow b^{1-cx} = -\frac{1}{c \log(b)}$$

$$\Leftrightarrow (1 - cx) \log(b) = \log\left(-\frac{1}{c \log(b)}\right)$$

$$\Leftrightarrow 1 - cx = \frac{\log\left(-\frac{1}{c \log(b)}\right)}{\log(b)}$$

$$\Leftrightarrow x = \frac{1}{c} - \frac{\log\left(-\frac{1}{c \log(b)}\right)}{c \log(b)}$$

Find a for which $f(x = a; b, c) = a$.

$$\frac{1}{\exp(\log(b)(cx-1))} = x$$

$$\Leftrightarrow (1 - cx) \log(b) = \log(x)$$

$$\Leftrightarrow (1 - cx) \log(b) - \log(x) = 0$$

```
In[337]:= Clear[x, b, c]
aval[x_, b_, c_] := Solve[(1 - c * x) * Log[b] - Log[x] == 0, x][[1]][[2]];
aval[x, b, c]
```

 **Solve:** Inverse functions are being used by Solve, so some solutions may not be found; use Reduce for complete solution information.

```
Out[339]= \frac{W(bc \log(b))}{c \log(b)}
```

$$a = \frac{W(c b \log(b))}{c \log(b)}$$

Set $x = a$

$$x = a$$

$$\Leftrightarrow \frac{1}{c} - \frac{\log\left(-\frac{1}{c \log(b)}\right)}{c \log(b)} = \frac{W(c b \log(b))}{c \log(b)}$$

$$\Leftrightarrow \log(b) - \log\left(-\frac{1}{c \log(b)}\right) = W(c b \log(b))$$

```
In[340]:= Clear[x, b, c]
Solve[Log[b] - Log[-1 / (c * Log[b])] == aVal[x, b, c] * c * Log[b], c][[1]][[1]][2]]
```

 **Solve:** Inverse functions are being used by Solve, so some solutions may not be found; use Reduce for complete solution information.

$$\text{Out[341]} = -\frac{1}{e b \log(b)}$$

$$c = -\frac{1}{e b \log(b)}$$

Plug in expression of c in $(1 - c x) \log(b) - \log(x) = 0$:

```
In[342]:= Clear[x, b, c]
aVal[x_, b_, c_] :=
  Solve[(1 - (-1 / (E * b * Log[b]))) * x) * Log[b] - Log[x] == 0, x][[1]][[1]][2]];
aVal[
  x,
  b,
  c]
```

 **Solve:** Inverse functions are being used by Solve, so some solutions may not be found; use Reduce for complete solution information.

$$\text{Out[344]} = e b$$

So we have that $a = e b$.

We can now plot the final version of h :

Downward limiter function, version 2a

$$f(x) = \frac{1}{\exp(\log(b)(c x - 1))}, c = -\frac{1}{e b \log(b)}$$

$$g(x) = \begin{cases} f(x) & \text{for } x < a \equiv e b, 0 \leq \lambda \leq 1 \\ x & \text{for } x > a \equiv e b \end{cases}$$

$$h(x) = \lambda g(x) + (1 - \lambda) \max(x, b)$$

```

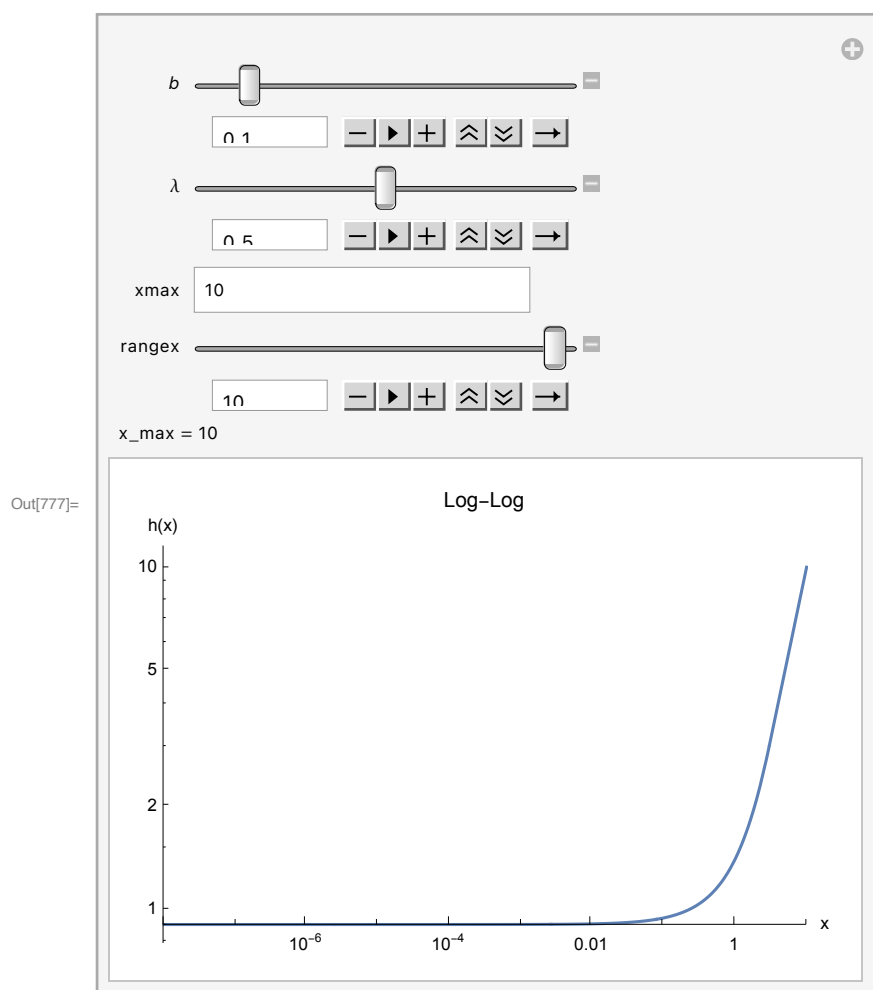
In[770]:=  $\delta = 1 \cdot 10^{-8}$ ;
xmaxinit = 10;
lambdainit = 0.5;

Clear[x, a, b]

f[x_, b_] := 1 / (Exp[Log[b] * ((- 1 / (E * b * Log[b])) * x - 1)]);
g[x_, b_] := Piecewise[
  {{f[x, b], x < E b},
  {x, x ≥ E b}}
];
h[x_, b_, λ_] := λ * g[x, b] + (1 - λ) * Max[b, x]

Manipulate[
  LogLogPlot[h[x, b, λ],
    {x, δ, rangex},
    AxesLabel → {"x", "h(x)"},
    PlotLabel → "Log-Log",
    PlotRange → {{δ, rangex}, Full}
  ],
  {{b, 0.1}, δ, 1 - δ, Appearance → "Open"},
  {{λ, lambdainit}, 0, 1, Appearance → "Open"},
  {xmax, xmaxinit},
  {{rangex, xmax}, δ, xmax, Appearance → "Open"},
  Dynamic[Style["x_max = " <> TextString[xmax]]]
]

```



We can apply the same idea to h -function version 1:

Downward limiter function, version 2b

```
ln[778]:=  $\delta = 1 \cdot 10^{-8}$ ;
bmax = 1;
cmax = 10;
binit = 0.1;
cinit = 0.1;
kinit = N[E+2];
lambdainit = 0.5;
(*xtreme=E+2;*) (*The higher the max x, the smaller the range of c*)
```

```
Clear[x, a, b, c]
```

```
(*aval[b_,c_,d_]:=1/c-log (d/(c log (b)))/(c log (b))**)
(*aval[x_,b_,c_]:=
  Solve[Log[b]*(1-c*x)-Log[x]==Log[-E*b*c Log[b]],x][[1]][[2]]*);
aval[b_,  $\gamma$ _]:= -1 / ( $\gamma$  Log[b]);
(*tauzero[b_,  $\gamma$ _]:=  $\gamma$  * b*Log[b]*E*);
(*tauinf[xtreme_, b_,  $\gamma$ _]:=  $\gamma$  * b*Log[b]*(E - b^(- $\gamma$ *xtreme))**)
```



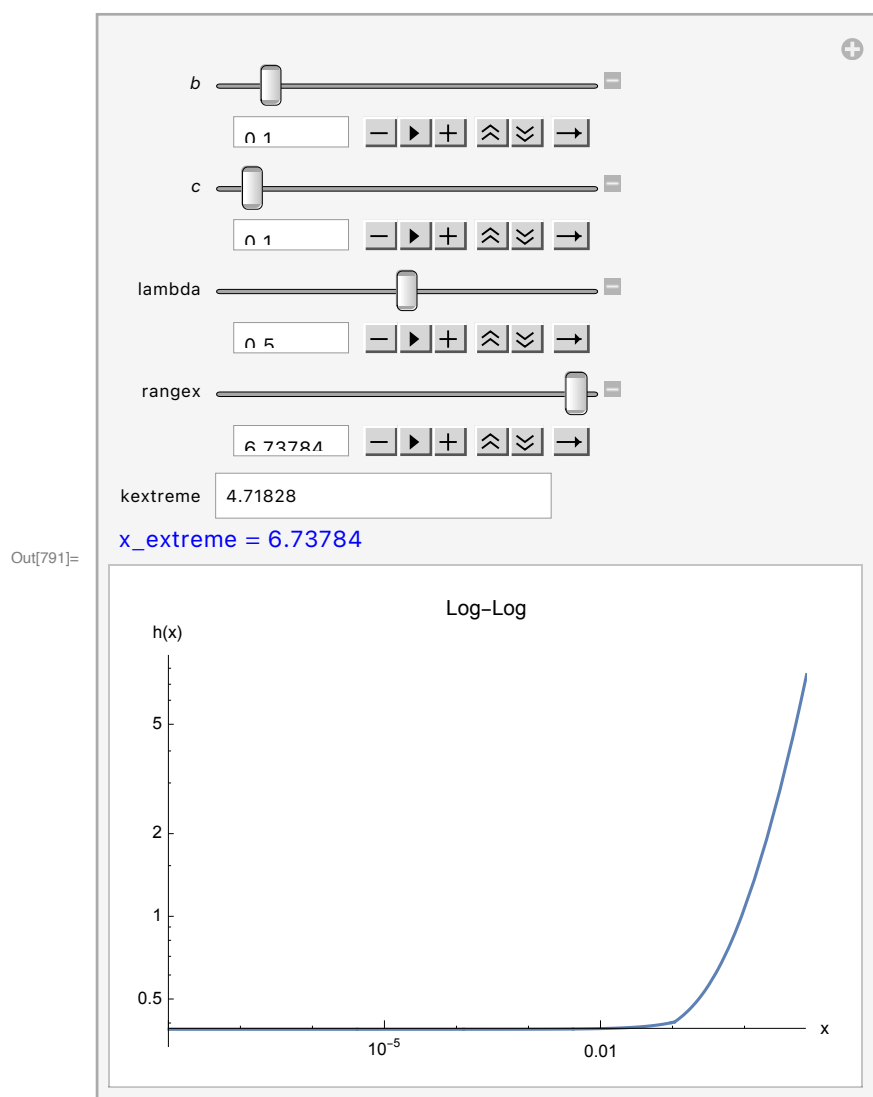
```

tauinf[kextreme_,  $\beta$ _,  $\gamma$ _] :=  $\gamma * \beta * \text{Log}[\beta] * (E - kextreme)$ 

f[x_,  $\beta$ _,  $\gamma$ _] := 1 / (Exp[Log[ $\beta$ ] * ( $\gamma * x - 1$ )] + (E *  $\gamma * \beta * \text{Log}[\beta] + 1) * x$ );
g[x_,  $\beta$ _,  $\gamma$ _] := Piecewise[{{f[x,  $\beta$ ,  $\gamma$ ], x < aval[ $\beta$ ,  $\gamma$ ]}, {g[x], x ≥ aval[ $\beta$ ,  $\gamma$ ]}];
h[x_,  $\beta$ _,  $\gamma$ _,  $\lambda$ _] :=  $\lambda * f[x, \beta, \gamma] + (1 - \lambda) * \text{Max}[x, \beta]$ ;

Manipulate[
  LogLogPlot[h[x, b, c, lambda],
    {x,  $\delta$ , rangex},
    AxesLabel → {"x", "h(x)"},
    PlotLabel → "Log-Log",
    PlotRange → {{ $\delta$ , rangex}, Full}
  ],
  {{b, binit},  $\delta$ , bmax -  $\delta$ , Appearance → "Open"},
  (*{{c, 0.1},  $\delta$ , If[b>1, 1/(b * Log[b] * E), cmax], Appearance → "Open"}*)
  (*{{c, 0.1},  $\delta$ , If[b>1, NSolve[tauzero[b,  $\gamma$ ] == 1,  $\gamma$ , Reals][[1]][[2]],
    NSolve[tauinf[b,  $\gamma$ ] == 1,  $\gamma$ , Reals][[1]][[2]], Appearance → "Open"}*)
  (*{{c, 0.1},  $\delta$ , Min[1/(b * Log[b] * (E - xtreme)), cmax], Appearance → "Open"},*)
  {{c, cinit},  $\delta$ ,
    Min[NSolve[tauinf[kextreme, b,  $\gamma$ ] == 1,  $\gamma$ , PositiveReals][[1]][[2]], cmax],
    Appearance → "Open"},
  {{lambda, lambdainit}, 0, 1, Appearance → "Open"},
  (*{{rangex, xtreme}, 2 *  $\delta$ , xtreme, Appearance → "Open"},*)
  {{rangex, -Log[kinit] / (cinit * Log[binit])},
    2 *  $\delta$ , -Log[kextreme] / (c * Log[b]), Appearance → "Open"},
  (*{xtreme, N[E+2]}*)
  {kextreme, kinit},
  Dynamic[Style["x_extreme = " <> TextString[N[-Log[kextreme] / (c * Log[b])]],
    12, If[kextreme > E, Blue, Red]]]
  (*{{c, 1/(2 * Log[b] * (b - 1))},  $\delta$ , Min[maxc, 1/(Log[b] * (b - 1)) -  $\delta$ ],
    Appearance → "Open"}*)
]

```



Version 2b offers more control, but is also much more difficult to calibrate and can be dangerously misspecified if not implemented with great care:

```

In[792]:=  $\delta = 1 \cdot 10^{-8}$ ;
bmax = 1;
cmax = 10;
binit = 0.1;
cinit = 10;
kinit = N[E + 2];
lambdainit = 0.5;
(*xtreme=E+2;*) (*The higher the max x, the smaller the range of c*)

```

```
Clear[x, a, b, c]
```

```

(*aval[b_,c_,d_]:=1/c-log (d/(c log (b)))/(c log (b))*
(*aval[x_,b_,c_]:=
Solve[Log[b]*(1-c*x)-Log[x]==Log[-E*b*c Log[b]],x][[1]][[2]];*)
aval[b_,  $\gamma$ ] := -1 / ( $\gamma$  Log[b]);
(*tauzero[b_,  $\gamma$ ] :=  $\gamma$  * b*Log[b]*E;*)

```

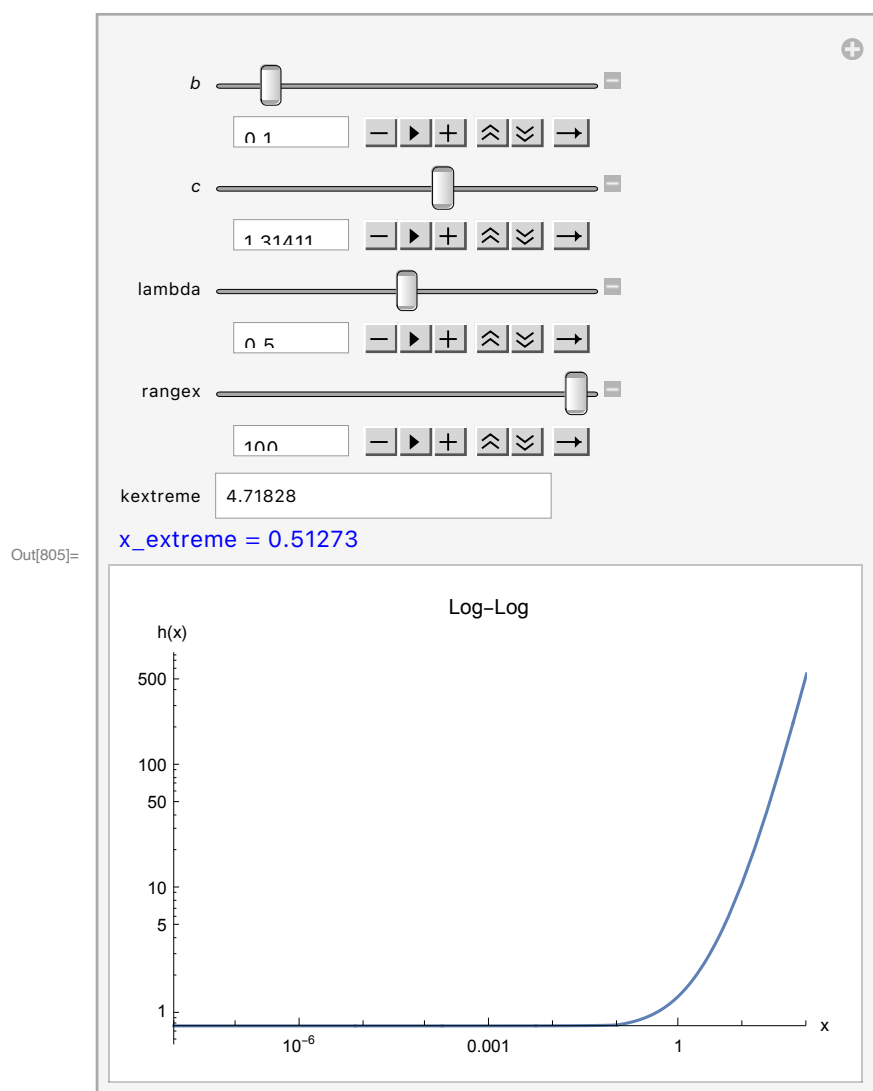
```

(*tauinf[xtreme_, b_, γ_] := γ * b*Log[b]*(E - b^(-γ*xtreme))*
tauinf[kextreme_, β_, γ_] := γ * β*Log[β] * (E - kextreme)

f[x_, β_, γ_] := 1 / (Exp[Log[β] * (γ*x - 1)] + (E * γ * β * Log[β] + 1) * x;
g[x_, β_, γ_] := Piecewise[{{f[x, β, γ], x < aval[β, γ]}, {g[x], x ≥ aval[β, γ]}}];
h[x_, β_, γ_, λ_] := λ*f[x, β, γ] + (1 - λ) * Max[x, β];

Manipulate[
  LogLogPlot[h[x, b, c, lambda],
    {x, δ, rangex},
    AxesLabel → {"x", "h(x)"},
    PlotLabel → "Log-Log",
    PlotRange → {{δ, rangex}, Full}
  ],
  {{b, binit}, δ, bmax - δ, Appearance → "Open"},
  (*{{c, 0.1}, δ, If[b>1, 1/(b *Log[b]* E), cmax], Appearance→"Open"}*)
  (*{{c, 0.1}, δ, If[b>1, NSolve[tauzero[b, γ]==1, γ, Reals][[1]][[1]][[2]],
    NSolve[tauinf[b, γ]==1, γ, Reals][[1]][[1]][[2]], Appearance→"Open"}*)
  (*{{c, 0.1}, δ, Min[1/(b*Log[b]*(E-xtreme)), cmax], Appearance→"Open"},*)
  {{c, cinit}, δ,
    Min[NSolve[tauinf[kextreme, b, γ] == 1, γ, PositiveReals][[1]][[1]][[2]], cmax],
    Appearance → "Open"},
  {{lambda, lambdainit}, 0, 1, Appearance → "Open"},
  (*{{rangex, xtreme}, 2*δ, xtreme, Appearance→"Open"},*)
  {{rangex, 100}, 2 * δ, 100, Appearance → "Open"},
  (*{xtreme, N[E+2]}*)
  {kextreme, kinit},
  Dynamic[Style["x_extreme = " <> TextString[N[-Log[kextreme] / (c * Log[b])]],
    12, If[kextreme > E, Blue, Red]]]
  (*{{c, 1/(2*Log[b]*(b-1))}, δ, Min[maxc, 1/(Log[b]*(b-1)) - δ],
    Appearance→"Open"}*)
]

```



Soft knee downward limiter function, version 3

The idea here is to connect the horizontal line $y = b$ with the “bypass” line $y = x$ by a suitable section of the line $y \equiv f(x; a, c) = a(x - c)^2$:

$$f(x; a, c) = a(x - c)^2$$

$$f'(x; a, c) = 2a(x - c)$$

The slope of f is 1 when $2a(x - c) = 1 \Leftrightarrow x = \frac{1}{2a} + c$

The slope of f is 0 when $2a(x - c) = 0 \Leftrightarrow x = c$.

$f(x; a, c)$ is a tangent to $y = x$, when

$$a(x - c)^2 = x \text{ and } x = \frac{1}{2a} + c$$

$$\Leftrightarrow a\left(\frac{1}{2a}\right)^2 = \frac{1}{2a} + c$$

$$\Leftrightarrow C = -\frac{1}{4a}$$

$$h(x; a, b) = \begin{cases} b & , \text{ for } 0 < x \leq b - \frac{1}{4a} \\ f(x; a) & , \text{ for } b - \frac{1}{4a} < x \leq b + \frac{1}{4a} \\ x & , \text{ for } x < b + \frac{1}{4a} \end{cases}$$

```

In[806]:= ainit = 0.2;
binit = 2;
bmax = 10;
bmax = 10;

δ = 10^(-8);

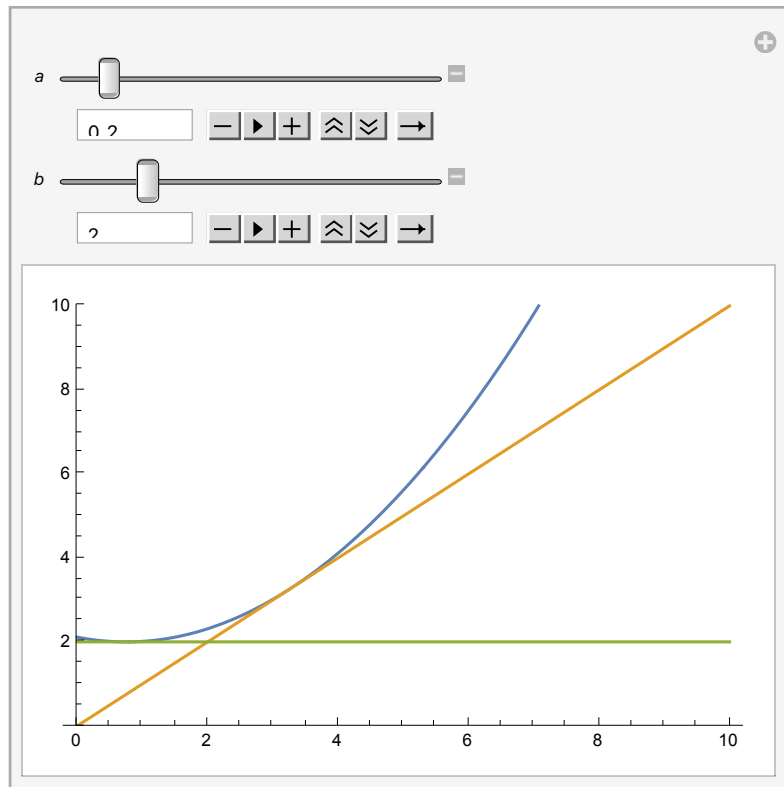
Clear[x, a, b, c]

f[x_, a_, b_] := a * (x - b + 1 / (4 * a))^2 + b;

Manipulate[
  Plot[
    {
      {f[x, a, b]},
      {x},
      {b}
    },
    {x, δ, bmax},
    PlotRange -> {0, 10}
  ],
  {{a, ainit}, 1 / (4 * b), amax, Appearance -> "Open"},
  {{b, binit}, δ, bmax, Appearance -> "Open"}
]

```

Out[813]=



```

In[814]:= ainit = 0.2;
binit = 2;
amax = 1;
bmax = 10;

 $\delta = 10^{-8}$ ;

Clear[x, a, b, c]

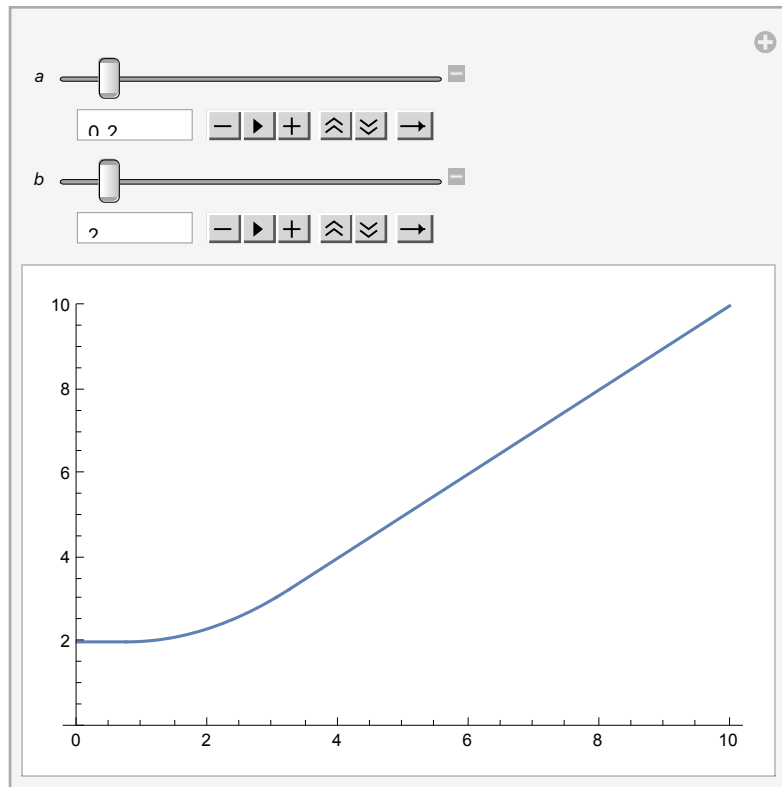
f[x_, a_, b_] := a * (x - b + 1 / (4 * a)) ^ 2 + b;

h[x_, a_, b_] := Piecewise[
{
  {b, x < b - 1 / (4 * a)},
  {f[x, a, b], b - 1 / (4 * a) ≤ x && x ≤ b + 1 / (4 * a)},
  {x, x > b + 1 / (4 * a)}
}
]

Manipulate[
  Plot[
    h[x, a, b],
    {x,  $\delta$ , bmax},
    PlotRange → {0, 10}
  ],
  {{a, ainit}, 1 / (4 * b), amax, Appearance → "Open"},
  {{b, binit}, 1 / (4 * a), bmax, Appearance → "Open"}
]

```

Out[822]=



Constraints

The constraints on the limiting multiplier are:

Constraints on x

$x \geq 0$ (because x is a standard deviation)

Constraints on a

Minimum value of a :

$$b > \frac{1}{4a} \Leftrightarrow a > \frac{1}{4b}$$

Constraints on b

$b > \frac{1}{4a}$ because otherwise the point where f touches $y = b$ will be below $x = 0$, and $f(x)$ will overshoot the limit of $f(0) = b$.

Derivation of the upward limiter function

We want to limit risk (standard deviation) upward.

Above $x = a$, limit y with a function, f , such that y approaches a limit, b , as x increases from below.

Below $x = a$, we want the input x to pass unchanged to the output y , i.e. $y = x$.

Let $g(x) = x$.

So we want to construct a piecewise function which is equal to f above $x = a$ and equal to g below $x = a$.

Initial idea for f :

$f(x)$ should tend to the threshold b , as x tends to ∞ .

$f'(x)$ should tend to 0, as x tends to ∞ .

This gives us the following idea:

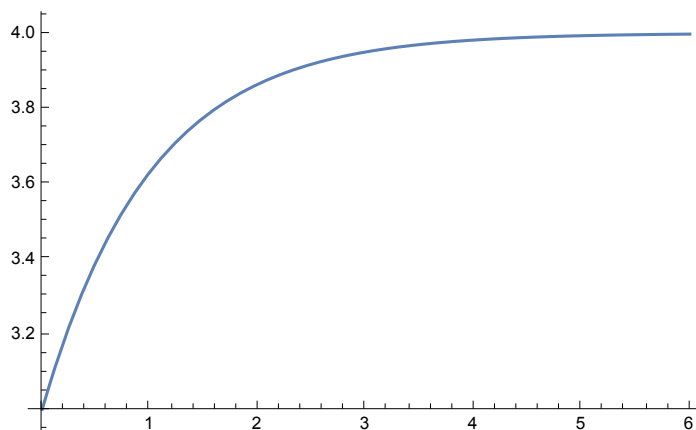
$$f(x) = b - \frac{1}{\exp(x)} = b - \exp(-x)$$

$$f'(x) = \frac{1}{\exp(x)} = \exp(-x)$$

In[823]:=

```
b = 4;  
Plot[  
  -Exp[-x] + b,  
  {x, 0, 6}  
]
```

Out[824]=

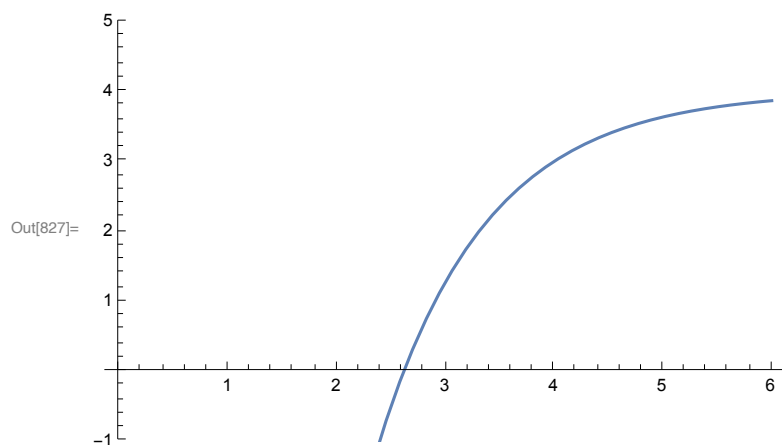


The plot suggests that we want to be able to shift the graph towards the right. For this we introduce a :

$$f(x) = b - \frac{1}{\exp(x-a)} = b - \exp(a-x)$$

$$f'(x) = \exp(a-x)$$

```
In[825]:= b = 4;
a = 4;
Plot[
  b - Exp[a - x],
  {x, 0, 6},
  PlotRange -> {-1, 5}
]
```

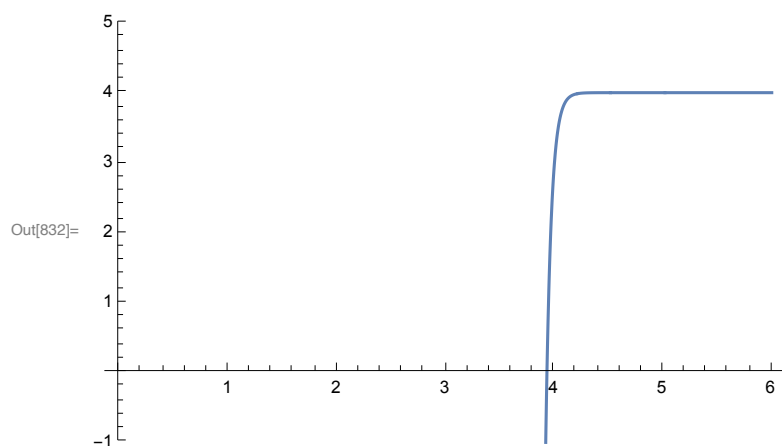


As for the downward limiter, we also want to be able to change the shape of the knee. For this we introduce the variable c :

$$f(x; a, b, c) = b - \frac{1}{\exp(c(x-a))} = b - \exp(c(a-x))$$

$$f'(x) = \exp(c(a-x))$$

```
In[828]:= b = 4;
a = 4;
c = 20;
k = 2;
Plot[
  b - Exp[c (a - x)],
  {x, 0, 6},
  PlotRange -> {-1, 5}
]
```

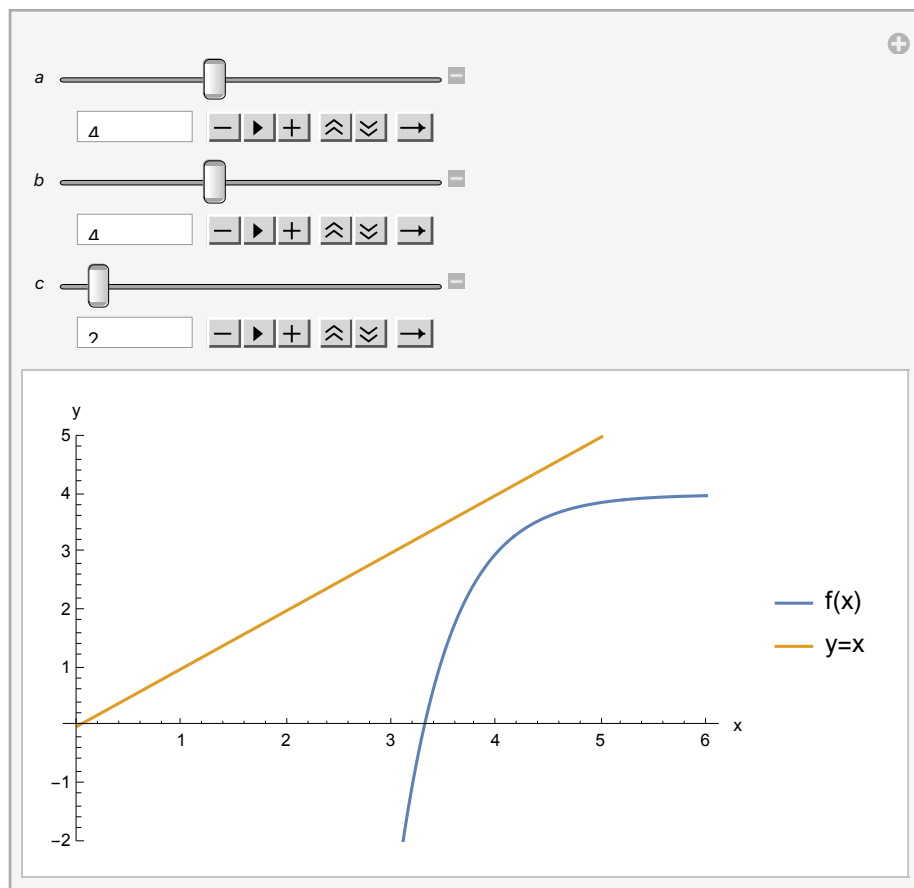


```

Clear[x, a, b, c]
(*f6temp[x_, b_, c_, d_] := 1/(Exp[Log[b]*(c * x - 1)]) + d*x*)
f7temp[x_, a_, b_, c_] := b - Exp[c (a - x)]
Manipulate[
  Plot[
    {f7temp[x, a, b, c], x},
    {x, 0, 6},
    PlotLegends -> {"f(x)", "y=x"},
    PlotRange -> {-2, 5},
    AxesLabel -> {"x", "y"}
  ],
  {{a, 4}, 0.1, 10, Appearance -> "Open"},
  {{b, 4}, 0.1, 10, Appearance -> "Open"},
  {{c, 2}, 1, 20, Appearance -> "Open"}
]

```

Out[835]=



Now we need to find the point where $f(a) = a$. At this point, g will touch f (though not necessarily be tangent). For this we modify f slightly:

$$f(x; a, b, c) = b - \frac{b-a}{\exp(c(x-a))}$$

$$f'(x) = (b-a) \exp(c(a-x))$$

We need to find the point where f' is 1 and $f = g$.

```
In[411]:= Clear[a, b, c]
          f7temp2[x_, a_, b_, c_] := b - (b - a) / Exp[c (x - a)]
          df7[x_] = D[f7temp2[x, a, b, c], x];
          df7[x]
```

```
Solve[df7[a] == 1, a]
```

```
Out[414]= c (b - a) e^{-c (x-a)}
```

```
Out[415]= {{a -> \frac{b c - 1}{c}}}
```

We see that $f'(x) = 1$ when $a = \frac{bc-1}{c}$.

We can now plug this expression of a into f , and join the piecewise functions at $x = \frac{bc-1}{c}$:

$$f(x; b, c) = b - \frac{b - \frac{bc-1}{c}}{\exp\left(c\left(x - \frac{bc-1}{c}\right)\right)}$$

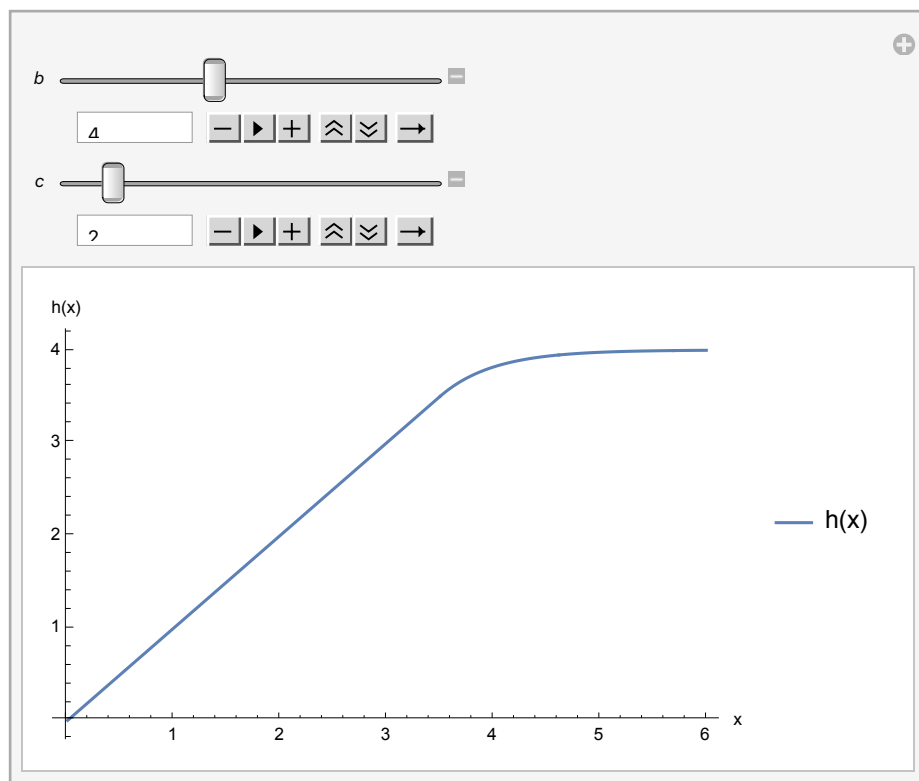
$$f'(x) = c\left(b - \frac{bc-1}{c}\right) \exp\left(c\left(\frac{bc-1}{c} - x\right)\right)$$

```
In[836]:= Clear[b, c, x]
```

```
f7[x_, b_, c_] := b - (b - (b * c - 1) / c) / Exp[c (x - (b * c - 1) / c)]
g7[x_] := x
h7[x_, b_, c_] :=
  Piecewise[{{f7[x, b, c], x > (b * c - 1) / c}, {g7[x], x ≤ (b * c - 1) / c}}]
```

```
Manipulate[
  Plot[
    h7[x, b, c],
    {x, 0, 6},
    PlotLegends → {"h(x)"},
    AxesLabel → {"x", "h(x)"}
  ],
  {{b, 4}, 0.1, 10, Appearance → "Open"},
  {{c, 2}, 0.1, 20, Appearance → "Open"}
]
```

Out[840]=



Constraints

And this works beautifully! We just need to find the valid parameter ranges for b and c .

These will be ones where f' is always positive for positive values of x .

So, we have $f'(x) = \left(b - \frac{bc-1}{c}\right) \exp\left(c\left(\frac{bc-1}{c} - x\right)\right)$, which gives us the constraint:

$$\left(b - \frac{bc-1}{c}\right) \exp\left(c\left(\frac{bc-1}{c} - x\right)\right) > 0$$

Initial constraints:

1) $x > 0$ (because x is a standard deviation)

2) $bc > 1$, because $a = \frac{bc-1}{c} \leq 0$ for $bc \leq 1$.

3a) $b > 0$ because we take log of b (because b is a risk value, i.e. a standard deviation).

3b) Given c , then $b > \frac{1}{c}$.

4a) Because $b > 0$, then $c > 0$, which follows from 2) and 3a).

4b) Given b , then $c > \frac{1}{b}$, which follows from 2) and 3).

$\exp\left(c\left(\frac{bc-1}{c} - x\right)\right)$ is always positive, so $f'(x) > 0$, when $b - \frac{bc-1}{c} > 0 \iff bc > bc - 1$.

So there are no other constraints than the initial ones.

Note on the product log function

To solve this problem we often run into situations where we need to solve equations of the general form

$$x e^x - 1 = 0.$$

This has a solution which can not be expressed with elementary functions. The solution is the product log function – or Lambert W function.

See https://en.wikipedia.org/wiki/Lambert_W_function

Scrap